
NegoPlay

From Bridge Tables to the Negotiation Table:
Do Decision-Making Styles Transfer Across Domains?

Final Project Portfolio

Final Summary Report · Preparatory Report · Business Plan

Submitted by: Anna Ben-Shushan

Lecturer: Dr. Yoram Segal

Course supervisor: Dr. Rami

Program: AI Development Expert Program

Institution: Tel Aviv University

Submission date: June 14, 2026

Executive Abstract

Real-world negotiation data is proprietary and unmeasured, which makes negotiation behaviour nearly impossible to study at scale. **NegoPlay** uses competitive bridge as a measurable laboratory for the same decisions — risk, pressure, and information exchange — and asks whether a player’s *decision-making style* transfers from bridge to business negotiation.

From 149,208 tournament records (563 qualifying players), an unsupervised pipeline discovered five statistically validated decision profiles (Cohen’s $d = 2.1\text{--}4.6$, all $p < 0.05$). Each profile was converted into an LLM agent whose system prompt contains only *bridge-derived* skills, and the agents negotiated business scenarios against a seller calibrated on 5,247 real Craigslist negotiations.

Win-rate transfer proved weak and metric-sensitive ($\rho \approx +0.2$); refining the question to its original behavioural form showed that **style transfers strongly**: bridge aggression and negotiation aggression correlate at **Spearman** $\rho = +0.80$, above the pre-registered 0.70 target. An inverse-prompt control flips the correlation to -0.90 , proving the transfer is carried by the bridge-derived skills rather than profile labels. The bridge metric itself was validated with a random-baseline (“monkey”) test and a double-dummy evaluator. The system is fully reproducible: a single SDK entry point, 100+ unit tests, seeded simulations, and per-call cost logging (total LLM spend $\approx \$1.70$).

Acknowledgments

I would like to thank the people who accompanied this project from idea to working system.

Dr. Yoram Segal, for the engineering-first project framework that shaped this work — the single-SDK discipline, the insistence on measurable deliverables, and the standard that a research prototype should still be a well-built piece of software.

Dr. Rami, for sharp methodological guidance: the demand for baselines, preprocessing audits, and statistical honesty pushed this project to validate every number it reports — including the random-baseline test that reshaped the bridge metric.

Prof. Jari Härmäläinen (LUT University), my PhD supervisor, for the academic home in which this project serves as the empirical baseline.

Dr. Nezer Jacob Zaidenberg, for bridge-domain expertise — from sample-size corrections in profile discovery to the double-dummy reference that anchored the skill metric.

And finally, my family — for the patience during late-night simulation runs and for pretending to find penalty doubles interesting.

Contents

Executive Abstract	1
Acknowledgments	2
I Final Summary Report	4
1 Project Overview	5
1.1 The problem and the idea	5
1.2 System pipeline	5
1.3 Data	5
2 Key Results	6
2.1 A trustworthy bridge metric (monkey + double-dummy)	6
2.2 The discovery: style transfers, winning does not	7
2.3 The anti-tautology control	7
2.4 Honest limitations	8
3 The AI Layer: LLM Agents and Skill Engineering	9
3.1 Agent architecture	9
3.2 Skill extraction and the anti-tautology rule	9
3.3 Prompt engineering as configuration	9
3.4 Engineering against non-determinism	9
3.5 Provider-neutral LLM client	9
4 Software Engineering	11
4.1 Architecture: a single SDK entry point	11
4.2 Testing	11
4.3 Reliability engineering	11
4.4 Reproducibility and data discipline	11
4.5 Code quality and cost as an engineering budget	12
5 Conclusions and Future Work	13
II Preparatory Report	14
Full preparatory report (51 pages): research question, methodology, mathematical foundations, results, schedule, work packages, risk management	
III Business Plan	66
Full business plan (20 pages): problem, solution, market analysis (TAM/SAM/SOM), business model, token economics, competitive landscape	

Part I

Final Summary Report

1 Project Overview

1.1 The problem and the idea

Business negotiation is one of the most consequential skills in industry, yet it is nearly impossible to study quantitatively: transcripts are confidential, outcomes are not standardised, and no public dataset links a person’s decision style to their negotiation behaviour. Competitive bridge offers a way around this. Every deal is a sequence of recorded decisions under uncertainty — how high to commit, when to take a risk, when to fight — and every decision has a measurable outcome. NegoPlay treats bridge as a *behavioural laboratory*: discover decision-making profiles in bridge data, embody them in LLM agents, and test whether the style they encode transfers to a different strategic domain.

1.2 System pipeline

The system is a four-stage hybrid pipeline (classical ML $\rightarrow$ LLM), connected by persisted file artefacts:

1. **Profile discovery (ML).** 149,208 EuroBridge records $\rightarrow$ 8 behavioural features per player $\rightarrow$ extreme-percentile profiling with binomial validation $\rightarrow$ 5 profiles (Slam Hunter, Insurance Player, Fighter, NT Specialist, Generalist).
2. **Skill extraction (LLM).** Real hands per profile are chunked and sent to the LLM, which returns 5–7 characteristic skills as structured JSON.
3. **Agent construction.** Each profile becomes an agent whose character card contains *only* the bridge-derived skills — never personality adjectives (the anti-tautology rule).
4. **Dual simulation and alignment.** The agents bid bridge deals and negotiate business scenarios; behaviour and outcomes in the two domains are correlated.

1.3 Data

- **Bridge:** 149,208 board records from the public EuroBridge championship database (2016–2025), scraped with a custom Python pipeline; 563 players pass the ≥ 50 -board reliability threshold.
- **Negotiation (external validation):** 5,247 real human price negotiations (Stanford Craigslist Bargains corpus), used to validate the behavioural features and to calibrate the simulated seller.

2 Key Results

2.1 A trustworthy bridge metric (monkey + double-dummy)

A skill metric must rank a random agent last. Our first, coarse bridge proxy failed this test — a random “monkey” bidder *outscored* every expert profile (0.54 vs. 0.25–0.51). Re-scoring each contract by whether it *actually makes* under double-dummy (perfect-play) evaluation collapses the monkey to last place (0.10):

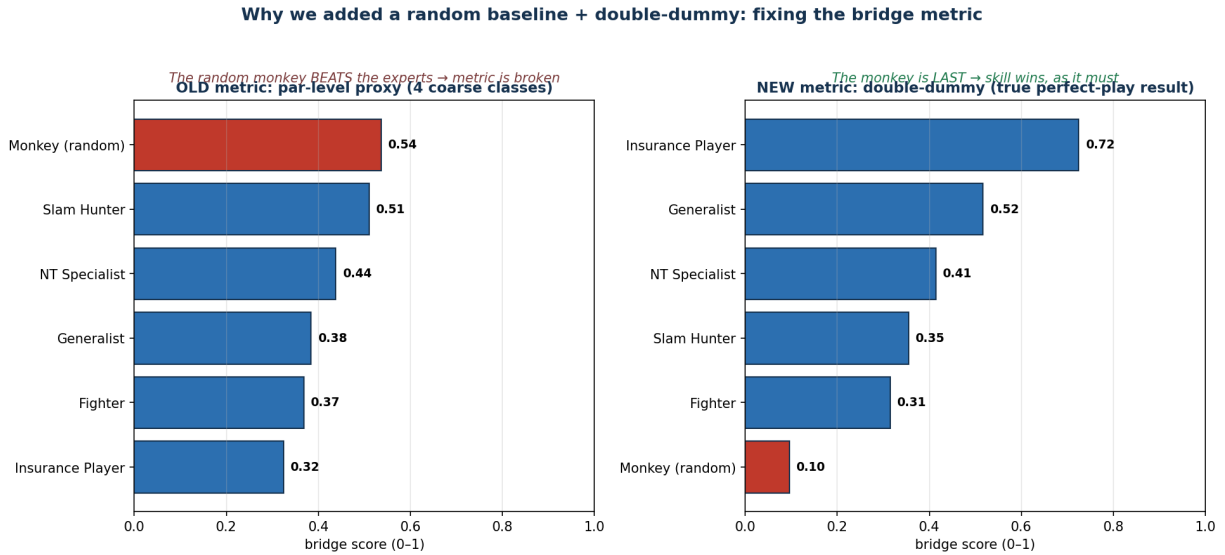


Figure 2.1: The random baseline exposes (left) and the double-dummy metric fixes (right) the bridge score.

Bridge skill was then measured from the *real* competitive results (duplicate IMP versus the field, defence included). On this scale the random monkey sits at -7 IMP per board, the elite profiles cluster near 0, and double-dummy perfect play marks the ceiling at $+1.1$:

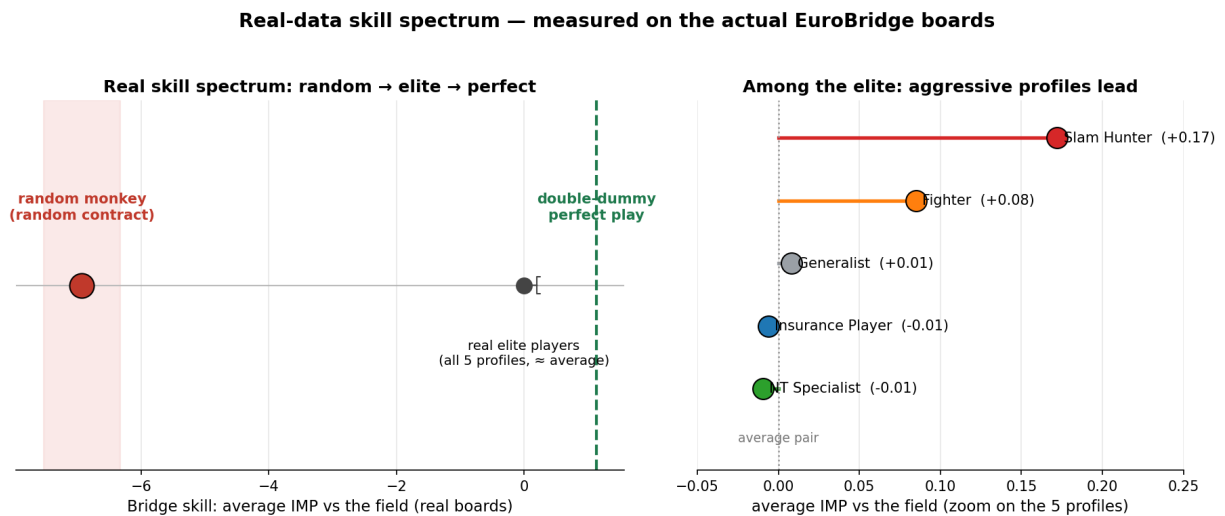


Figure 2.2: Real-data skill spectrum: random floor → elite players → perfect-play ceiling.

2.2 The discovery: style transfers, winning does not

Correlating *winning* across domains proved weak and metric-sensitive ($\rho \approx +0.2$ to $+0.5$), because whether a style wins depends on the opponent and the payoff rules. Refining the question to its original behavioural form — does an aggressive bridge profile *bargain* aggressively? — yields **Spearman** $\rho = +0.80$:

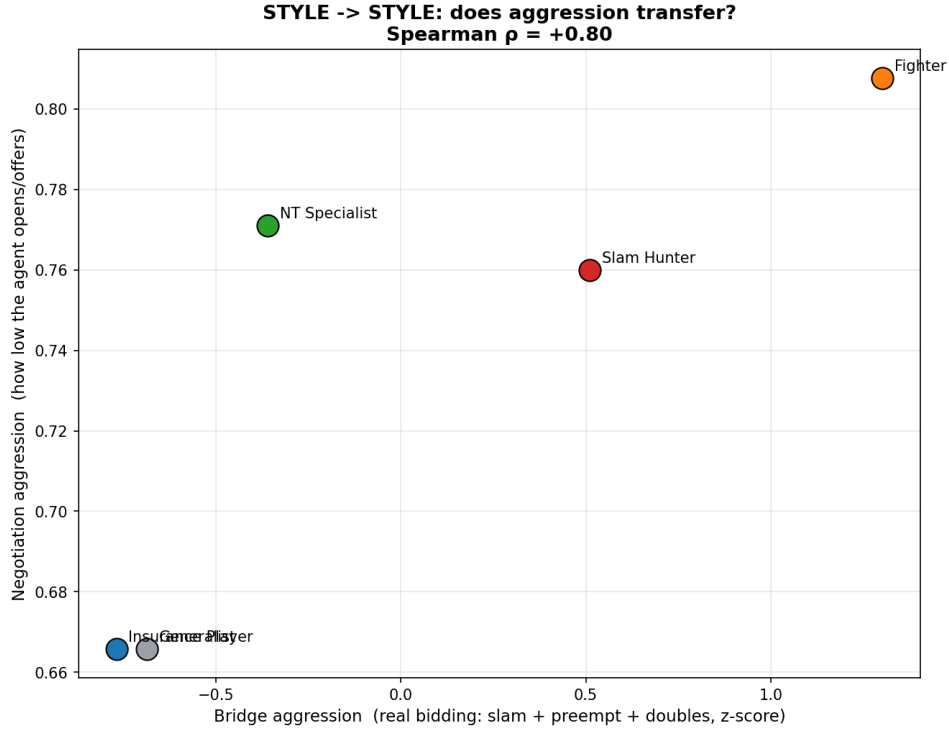


Figure 2.3: Style $\leftrightarrow$ style: bridge aggression (real bidding) vs. negotiation aggression (how low the agent opens), $\rho = +0.80$.

2.3 The anti-tautology control

To rule out “the agent is aggressive because we labelled it aggressive,” each profile kept its identity but received the *opposite* profile’s bridge skills. The correlation flipped from $+0.80$ to -0.90 ($p = 0.037$) — behaviour follows the injected bridge-derived skills, not the label:

Style transfers — and it is the SKILLS that carry it (anti-tautology control)

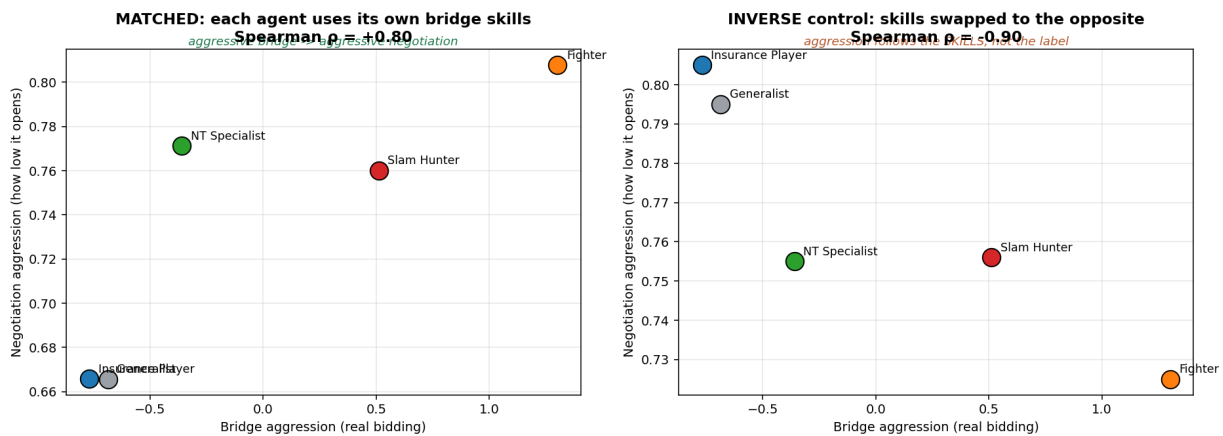


Figure 2.4: Matched skills (left, +0.80) vs. swapped skills (right, -0.90): the skills carry the style.

2.4 Honest limitations

With $n = 5$ profiles the style correlation is indicative ($p \approx 0.10$), not significant; negotiation is simulated (calibrated on real human negotiations, but no same-person negotiation data exists); and the Slam Hunter profile, defined by *bidding* slams, may partly proxy overall playing strength. These are documented, not hidden.

3 The AI Layer: LLM Agents and Skill Engineering

3.1 Agent architecture

All agents inherit from an abstract `BaseAgent` that owns the profile signature, the LLM client, and a single decision choke-point. No agent calls a provider SDK directly — every decision flows through one method, which centralises retries, JSON parsing, temperature policy, and cost logging:

```
class BaseAgent(ABC):
    def _decide(self, user_prompt, response_schema, purpose) -> dict:
        """The ONLY path to the LLM: schema-enforced, logged, retried."""
        response = self.client.generate(
            system=self.system_prompt, user=user_prompt,
            purpose=purpose, temperature=self.temperature,
            response_schema=response_schema)
        return response.json or json.loads(response.text)
```

Two concrete agents implement the domains: `BridgeAgent.make_bid()` (temperature 0 for reproducibility; every returned call is validated by a local, non-LLM legality checker and falls back to `Pass` if illegal) and `NegotiationAgent.respond_to_offer()` (temperature 0.7; returns `counter/accept/walk_away` with a price clamped to the scenario’s legal range).

3.2 Skill extraction and the anti-tautology rule

Stage 2 turns raw hands into a *skill profile*: hands are chunked (20–30 per call), sent with a strict JSON schema (`response_mime_type="application/json"`), and aggregated across chunks. The cardinal engineering rule: agents are **never** given personality adjectives. The Fighter’s card says “*doubles opponents at 6–8× the field rate*” — a fact extracted from its players’ real auctions — not “*be aggressive.*” Any cross-domain behaviour must therefore *emerge* from bridge-derived skills. The inverse-prompt control (Chapter 2) is the executable test of this rule: swapping the skills swaps the behaviour ($\rho : +0.80 \rightarrow -0.90$).

3.3 Prompt engineering as configuration

Prompts are code-reviewed artefacts, not ad-hoc strings: a central library (`shared/prompts.py`) builds every character card from the same template — identity line, 5–7 extracted skills, domain rules, a strict output schema, and a few-shot example. Temperature, schema, and provider are parameters, so an experiment is a configuration change, not a rewrite.

3.4 Engineering against non-determinism

LLMs are non-deterministic even at temperature 0 (parallel GPU execution). The system therefore treats reproducibility at the *sample* level: every (profile, deal) is run three times with majority voting; every raw response is persisted to JSONL before aggregation; and all analyses are recomputable from the persisted artefacts without new API calls. Deals themselves come from a seeded generator, so the entire experiment replays bit-identically up to the LLM boundary.

3.5 Provider-neutral LLM client

A single `LLMClient` wraps Gemini (default), Claude, and OpenAI behind one interface (an Adapter pattern). It implements exponential-backoff retries, a per-request 60-second HTTP time-

out (a hung socket once stalled a 1,500-call batch — the timeout converts hangs into retryable failures), and a JSONL cost log per call (provider, model, tokens, latency, USD). Total project LLM spend: $\approx$ \$1.70.

4 Software Engineering

4.1 Architecture: a single SDK entry point

The codebase follows a strict facade discipline: every public operation is reachable through `src/sdk.py`, and no function lives only in a notebook or script. Modules are organised by pipeline stage:

```
src/  
  sdk.py # facade: the single public contract  
  stage1_clustering/ # features, profiling, validation  
  stage2_skills/ # chunker, extractor, aggregator  
  stage3_agents/ # BaseAgent, BridgeAgent, NegotiationAgent  
  stage4_simulate/ # bridge_game, bridge_auction, negotiation,  
                  # monkey_agent, alignment  
  features/double_dummy.py # DDS perfect-play evaluator (endplay)  
  shared/ # llm_client, prompts, data_loader, logger
```

Design patterns in active use: **Facade** (`sdk.py`), **Template Method** (`BaseAgent._decide()`), **Adapter** (provider-neutral `LLMClient`), and **Strategy** (interchangeable profile agents, including the `MonkeyAgent` baseline, which is duck-typed to drop into every runner unchanged).

4.2 Testing

The project ships 100+ `pytest` tests: pure-logic units (deal generation, scoring, auction legality — 28 tests for the bridge engine alone), the random-baseline agent (legality of every random call, reproducibility under a fixed seed), the double-dummy evaluator (known fixtures: a 12-trick deal must score $6NT = 1.0$ and $7NT = 0.0$), and the LLM-dependent layers tested by *injecting a mocked client* — no network, no API key, no cost:

```
def test_monkey_bids_are_always_legal_over_many_auctions():  
    monkey = MonkeyAgent(seed=1)  
    for auction in AUCTION_FIXTURES:  
        for _ in range(50):  
            out = monkey.make_bid(HAND, auction)  
            assert is_legal_call(out["bid"], auction)
```

4.3 Reliability engineering

The long simulation runs are engineered to survive real-world failure modes observed during the project: a per-request HTTP timeout (hangs → retries), per-board *atomic* retry with pauses (rides out provider 503 load spikes without losing the batch), and incremental JSONL checkpointing with `flush()` after every board — a killed run keeps all completed work and progress is observable from the file system.

4.4 Reproducibility and data discipline

Fixed seeds (`random_state=42`) across scikit-learn and the deal generator; raw data immutable (`data/raw` is read-only by convention); every derived number traceable to a persisted artefact; and the full history in Git under conventional-commit discipline (`feat(stage4): ...`, `docs(guide): ...`). Every figure in this portfolio is generated by a committed script — nothing is hand-drawn, so documentation cannot drift from the data.

4.5 Code quality and cost as an engineering budget

Style and typing are enforced with **ruff**, type hints, and Google-style docstrings on all public functions. LLM cost is treated as an engineering budget: model selection is empirical (**gemini-2.5-flash-lite** proved 10× faster than the default for this workload), every call is metered, and the complete project — all stages, reruns included — cost ≈\$1.70.

5 Conclusions and Future Work

Conclusions. (1) Elite bridge players form a behavioural continuum, not clusters; statistically validated profiles live in its extreme tails. (2) A skill metric that a random agent can beat is broken — the random-baseline test materially changed this project’s measurements. (3) Decision *style* transfers across domains ($\rho = +0.80$, non-tautological by an executed inverse-prompt control); *winning* does not transfer cleanly, because winning is a property of the (style $\times$ environment) pair.

Future work. More profiles (finer partitioning of the 563 players) for statistical power; multi-dimensional negotiation-style measurement (concession speed, walk-away propensity); cross-model replication (Claude/GPT); separating style from playing strength; and ultimately real same-person negotiation data.

Part II

Preparatory Report

AI Development Expert – Final Project

Preliminary Report

NegoPlay

*Strategic Profiles from Bridge to Business:
An Empirical Investigation of Decision-Making Styles
using LLM Agents*

Student: Anna Ben-Shushan

Supervisor: Dr. Yoram Segal

Course: AI Development Expert

Document: Preliminary Project Report

Version: 1.0

Date: June 10, 2026

Acknowledgements

I would like to thank **Dr. Yoram Segal**, my project supervisor, for his structured methodology framework – in particular the requirement of clearly identified anchor papers, single-SDK architecture, and explicit baseline comparisons. These principles shaped this project from the first day.

This project is also the empirical baseline for my PhD research at LUT University (Finland), under the supervision of **Prof. Jari Hämmäläinen**. I gratefully acknowledge **Dr. Nezer Jacob Zaidenberg**, my PhD supervisor and advisor on bridge and cybersecurity matters, for ongoing discussions throughout this work.

The 149,208-row dataset used in this work was collected from the public EuroBridge database (European Bridge League, 2016–2025) and processed using a custom Python pipeline developed during the first weeks of 2026.

Contents

1	Introduction	6
1.1	Background and Motivation	6
1.2	Why this matters	6
1.3	Literature Review and Positioning	6
2	Research Question and Objectives	9
2.1	Primary Research Question	9
2.2	Hypotheses	9
2.3	Objectives	9
2.4	Anchor papers	9
3	Methodology	10
3.1	Overview	10
3.2	Dataset	10
3.3	Stage 1 – Profile Discovery (ML)	11
3.4	Stage 2 – Skill Extraction (LLM)	11
3.5	Stage 3 – Agent Construction	15
3.6	Stage 4 – Dual Simulation and Alignment	16
3.7	Tools and Stack	16
4	Technical Architecture and Mathematical Foundations	19
4.1	System Architecture	19
4.2	Stage 1: Mathematical Pipeline	19
4.2.1	Notation	20
4.2.2	Z-score standardisation	20
4.2.3	Variance audit (coefficient of variation)	21
4.2.4	Principal Component Analysis	21
4.2.5	Clustering validity: Silhouette score	22
4.2.6	Mahalanobis-distance outlier detection	22
4.2.7	Extreme-percentile profile assignment	23
4.3	Stage 3: LLM Agent Architecture	24
4.3.1	System prompt anatomy	24
4.3.2	Temperature sampling	25
4.3.3	Per-call cost accounting	25
4.4	Stage 4: Dual Simulation and Alignment Measure	25
4.4.1	Per-profile win rate	26
4.4.2	Spearman rank correlation	26
4.4.3	Falsifiability test (Phase 3)	27

5 Results: Stage 4 Cross-Domain Alignment	28
5.1 Step 1 (winning): building a trustworthy bridge metric	28
5.2 Measuring bridge from the real competitive data	29
5.3 Winning alignment: weak and metric-sensitive	29
5.4 The skill spectrum: random floor to perfect ceiling	30
5.5 Step 2 (style): does the decision STYLE transfer?	31
5.6 Anti-tautology control	32
5.7 Interpretation and caveats	32
6 Schedule	33
7 Work Packages	34
8 Gantt Chart and Dependencies	36
8.1 WP $\times$ Session matrix	36
8.2 Dependencies and the critical path	37
8.3 High-level Gantt	38
9 Deliverables and Success Criteria	39
9.1 How will we know we succeeded?	39
9.2 Deliverables	39
9.3 Engineering quality gates	40
10 Risk Management	41
10.1 Risk register	42
10.2 Risk monitoring	43
11 Summary	44
11.1 What this report covers	44
11.2 Expected contribution	44
11.3 Beyond the course: link to the PhD	44
A Reproducibility Package	45
A.1 Code repository	45
A.2 Dataset	45
A.3 Software environment	45
A.4 Stage 1 hyper-parameters	46
A.5 Population baselines (Stage 1 output)	46
A.6 Stage 2 – LLM call configuration	46
A.7 Stage 3 – Agent system prompts (template)	47
A.8 Stage 4 – Simulation protocol	47
A.9 Anchor papers	48
A.10 How to reproduce, end-to-end	48
A.11 Tests and continuous validation	48

List of Figures

3.1	The four-stage NegoPlay pipeline. Stages 1 and 4 produce the empirical evidence; stages 2 and 3 produce the agents.	10
3.2	Stage 2 validation visualised (563 players). Generated by <code>notebooks/visualize_for_supervisor.py</code> .	
4.1	NegoPlay system architecture. Solid arrows: information flow between stages. Dashed arrows: persisted artefacts on disk. All five stages are complete: the full pipeline ran end-to-end from the 149K-row raw dataset to the cross-domain alignment result (Chapter 5).	19
4.2	Stage 1 pipeline: raw data $\rightarrow$ feature engineering $\rightarrow$ statistical filters $\rightarrow$ scaling $\rightarrow$ PCA $\rightarrow$ extreme-percentile profile assignment. Each filter is labelled with its threshold.	20
4.3	LLM agent architecture: game state $\rightarrow$ prompt builder $\rightarrow$ Gemini $\rightarrow$ JSON response $\rightarrow$ validator $\rightarrow$ action. Invalid responses trigger a retry with the validation error in the next turn. Every call is logged with token counts and cost for reproducibility.	24
4.4	Stage 4 dual simulation: the same agents play bridge and negotiation, producing two vectors of profile-level win-rates. Spearman correlation between them is the main outcome.	26
5.1	A random baseline exposes, and a double-dummy metric fixes, the bridge score. <i>Left</i> : on the coarse proxy the random monkey (red) beats every profile. <i>Right</i> : scored by whether the contract actually makes (double-dummy), the monkey is last and skill wins.	28
5.2	Real competitive bridge skill (IMP, x) versus simulated negotiation surplus (y). Spearman $\rho = +0.50$; the aggressive profiles lead in both domains.	30
5.3	Real-data skill spectrum on the actual boards. <i>Left</i> : random monkey (-7 IMP) $\rightarrow$ elite players (≈ 0) $\rightarrow$ double-dummy perfect ($+1.1$ IMP). <i>Right (zoom)</i> : among the elite, the aggressive profiles (Slam Hunter, Fighter) lead.	31
5.4	<i>Left (matched)</i> : bridge aggression vs negotiation aggression, $\rho = +0.80$ – the style transfers. <i>Right (inverse control)</i> : when each profile is given the <i>opposite</i> profile’s bridge skills, the correlation flips to $\rho = -0.90$ – negotiation aggression follows the injected skills, not the label.	31

List of Tables

1.1	NegoPlay positioning against representative prior work.	8
3.1	NegoPlay dataset summary. Rows are progressively filtered as we narrow from the raw scrape to the analysis cohort actually used by Stage 1.	10
3.2	Stage 2 empirical validation: each profile’s defining behaviour vs. the Generalist baseline (validation sample, five players per profile).	12
5.1	Real competitive bridge skill (IMP versus the field, defence included) and the simulated negotiation surplus, per profile.	29
6.1	Five-week sprint plan.	33
8.1	Work-package matrix. Each row is one task; columns are the ten project sessions. Dark orange = critical / blocking. Light blue = non-blocking.	37
8.2	High-level Gantt by sprint.	38
9.1	Research key performance indicators.	39
10.1	Top project risks and mitigations.	42
A.1	Stage 1 hyper-parameters (production pipeline, May 2026).	46
A.2	NegoPlay population baselines and assigned-profile rates (v2.1, May 2026). . . .	46

1 Introduction

1.1 Background and Motivation

Decision making under incomplete information is one of the hardest problems in artificial intelligence. While modern AI has reached super-human performance in fully observable games such as chess (Silver et al., 2017), Go, and Atari (Schrittwieser et al., 2020), real-world strategic environments – business negotiation in particular – are characterized by hidden information, partial communication, and a mix of cooperation and competition that is hard to model directly.

Bridge is a unique testbed for these challenges. Each of the four players sees only 13 of the 52 cards, must cooperate with a partner while competing against two opponents, and communicates through a highly structured bidding language with approximately 10^{47} possible sequences. Bridge bidding is, formally, a multi-agent negotiation under imperfect information with measurable outcomes. Rong et al. (2019) showed that a deep learning system splitting bidding into estimation (ENN) and policy (PNN) networks can beat the world-leading rule-based engine Wbridge5; Brown and Sandholm (2019) demonstrated that imperfect-information AI can defeat elite humans in multi-player poker.

NegoPlay asks a more applied question: *can we extract decision-making profiles from a large dataset of elite bridge play, and reuse those profiles to predict and simulate behavior in business negotiation scenarios?*

1.2 Why this matters

Business negotiation suffers from a fundamental data scarcity problem. Real negotiation transcripts are proprietary, outcomes are rarely measurable in standardized form, and no public datasets allow training of behavioral models. Bridge, in contrast, offers a clean laboratory:

- More than 149,000 structured decision points with measurable outcomes.
- Clear win/loss signals (contract success, IMP scores, VP conversion).
- Tournament-grade decisions by expert players, with explicit player identity per seat.

If profiles discovered in this clean environment transfer measurably to the noisier domain of negotiation, this opens a methodological path for studying business strategy without proprietary negotiation data.

1.3 Literature Review and Positioning

This section grounds NegoPlay in the wider research conversation. Following Lockett et al. (2007), we treat an unknown counterpart as a mixture of *cardinal strategies* that can be discovered and re-used. We organize the literature into five themes.

Theme 1: Player-style clustering from gameplay sequences

Our methodological anchor is [Talwadder et al. \(2022\)](#), who introduced **CognitionNet**. Using a sequence-to-sequence interpreter coupled to a classifier, they discovered behavioral profiles directly from raw Rummy telemetry. NegoPlay replaces “Rummy telemetry” with “bridge bidding sequences” but keeps the same logic: *styles emerge from sequences of actions, without predefined rules*. [Szyber-Betley et al. \(2023\)](#) provide an alternative path – learning a continuous 8-dimensional embedding of bridge hands – that we may incorporate in a second iteration.

Theme 2: Inference + decision split (theory of mind)

Our second anchor is [Rong et al. \(2019\)](#), whose ENN/PNN architecture separates the inference of partner’s hidden cards from the action policy. [Wang et al. \(2024\)](#) extend this with a conditional GAN inference module. NegoPlay applies the same split at the LLM level: the cluster profile is the *inference* (“what kind of agent am I facing?”), and the system prompt is the *policy* (“how do I act?”). This theory-of-mind-before-action structure is now standard in published bridge AI.

Theme 3: Opponent as mixture of cardinal styles

[Lockett et al. \(2007\)](#) is the theoretical foundation for the entire project. Their result – that explicit mixture coefficients over a small set of cardinal opponent types outperform monolithic networks – justifies the central NegoPlay premise: every negotiation counterpart can be treated as a weighted mixture of the few profiles discovered in the bridge data.

Theme 4: Bridging structured games and natural-language negotiation

[James et al. \(2021\)](#) encode both actions and “cheap talk” messages into unified attitude vectors, then cluster them. This is the conceptual glue between the bridge stage (structured signals) and the negotiation stage (free-form text), and it is the paper we cite when defending the transfer claim.

Theme 5: Existing bridge AI engines as benchmarks

[Yeh et al. \(2018\)](#) and [Kita et al. \(2024\)](#) train deep RL bidders that beat Wbridge5; the open-source BEN engine ([lorserker, 2023](#)) provides a neural-network-based, scriptable opponent that we can use to ground LLM-vs-LLM games against an objective player. [Jarosz \(2022\)](#) provides a clean survey of the field.

What others did, and what NegoPlay does differently

Table 1.1: NegoPlay positioning against representative prior work.

Reference	What they did	What NegoPlay does differently
Talwadker et al. (2022)	Clustered Rummy players from telemetry sequences.	Cluster bridge <i>bidding</i> sequences; then <i>reuse</i> the profiles in a second, different domain (negotiation).
Rong et al. (2019)	Built a deep ENN/PNN bidding system that beats Wbridge5.	Replace the deep networks with <i>LLM agents</i> prompted by the discovered profile; trade peak strength for explainability and cross-domain reuse.
Lockett et al. (2007)	Evolved explicit opponent models using NEAT.	Replace the evolved cardinal set with a <i>data-driven, statistically validated</i> cardinal set (K-means + HDBSCAN).
James et al. (2021)	Clustered attitudes from actions + cheap talk in toy games.	Apply the same logic to a real, large-scale, expert dataset – and explicitly test transfer between two domains.

2 Research Question and Objectives

2.1 Primary Research Question

Can LLM agents built from automatically-discovered bridge decision-making profiles exhibit behavioral consistency (Spearman $\rho \geq 0.7$) between winning in bridge bidding simulations and winning in parallel business negotiation simulations?

2.2 Hypotheses

H1 – Statistical: K-means clustering on five behavioral features will produce 3–5 stable profiles with Silhouette score ≥ 0.4 and $p < 0.05$ against random clustering.

H2 – Behavioral: LLM agents instantiated with these profiles will exhibit measurably different behaviors in bridge bidding (per-profile win-rate variance $> 15\%$).

H3 – Transfer (main): The win-rate ranking across profiles will correlate $\rho \geq 0.7$ (Spearman) between the bridge and negotiation domains.

Null: No correlation > 0.5 between domains. This too is a valid research finding.

2.3 Objectives

The project has four operational objectives:

1. **Statistical:** discover 3–5 stable decision profiles from $\approx 1,500$ elite players.
2. **Linguistic:** translate each profile into a 5–7 trait skill description using an LLM.
3. **Engineering:** build four LLM agents (one per profile) with verifiable, structured output.
4. **Empirical:** run dual simulations (bridge + negotiation) and measure cross-domain alignment.

2.4 Anchor papers

Following Dr. Yoram Segal’s methodology framework, every project declares its anchor papers explicitly:

- **Anchor 1 (Stage 1 – clustering):** [Talwadker et al. \(2022\)](#).
- **Anchor 2 (Stage 3 – inference/decision split):** [Rong et al. \(2019\)](#).
- **Theoretical foundation:** [Lockett et al. \(2007\)](#).

3 Methodology

3.1 Overview

NegoPlay is a four-stage hybrid pipeline that combines classical machine learning with large language models (see Figure 3.1).

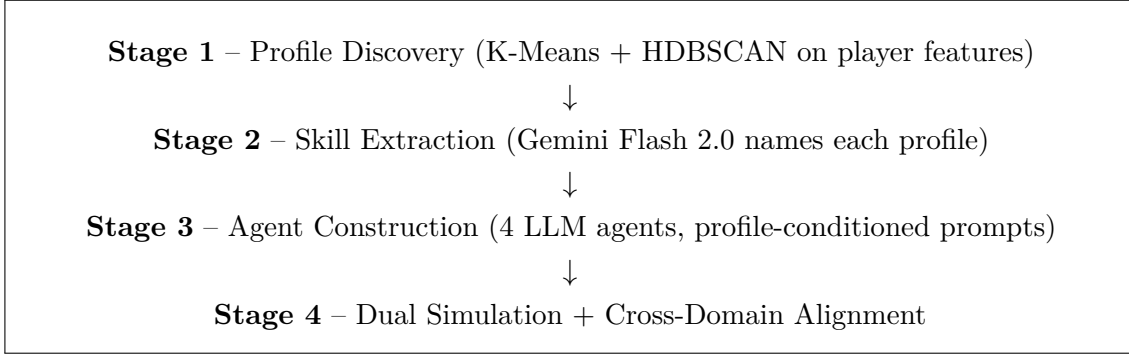


Figure 3.1: The four-stage NegoPlay pipeline. Stages 1 and 4 produce the empirical evidence; stages 2 and 3 produce the agents.

3.2 Dataset

The dataset was collected by the author from the public EuroBridge database between January and May 2026, using a custom Python scraping pipeline.

Table 3.1: NegoPlay dataset summary. Rows are progressively filtered as we narrow from the raw scrape to the analysis cohort actually used by Stage 1.

Metric	Value
Total rows (board $\times$ room)	149,208
European Championships covered	5 (2016–2025)
Unique players (by name)	$\approx 2,500$
Player-name coverage (N/S/E/W $\times$ open/closed)	99.9 %
Card-holding coverage (52 cards)	54 %
Bidding-sequence coverage	31 % (100 % on Herning 2024 + Poznan 2025)
<i>Analysis cohort – progressive filtering for Stage 1:</i>	
Players with ≥ 20 declared boards (legacy threshold)	1,572
Players with ≥ 50 declared <i>and</i> ≥ 50 bidding boards (production)	563

Why two thresholds appear. The legacy threshold of 20 boards was used in the v2.0 Stage 1 pipeline (April 2026), which produced 64 “Slam Hunters” out of 1,572 qualifying players. A bridge-domain expert review identified a sample-size problem: with only 20 boards and a rare-event rate near 5 %, the 95 % confidence interval is too wide to distinguish a true Slam Hunter from random variation. The pipeline was revised to require ≥ 50 boards on *both* the declared and the bidding axis, plus a one-sided binomial test (Eq. (4.11)). The cohort that enters all

numerical results in this report is therefore the **563-player** v2.1 cohort; the 1,572 figure is shown only as a reference for the underlying data scale. The full revision is documented in RESEARCH_INSIGHTS.md, sections Q7.5–Q7.6.

3.3 Stage 1 – Profile Discovery (ML)

Ten behavioural features are computed per player and partitioned into two groups: *outcome features* extracted from the `contract` column (any board the player declared) and *process features* extracted from the `bidding` column (any board where the auction is recorded, regardless of who declared). The process / outcome split follows the methodological anchor of [Rong et al. \(2019\)](#), who separate the inference (auction-process) signal from the action (outcome) signal.

- **Outcome features (declarer only, 5):** `slam_rate` (% of declarations at slam level 6–7); `double_rate` (% of declared contracts that opponents doubled); `avg_level` (mean contract level $\in [1, 7]$); `nt_rate` (% of declarations in NoTrump); `partscore_rate` (% of declarations at level 1–3).
- **Process features (any seat, from full bidding, 5):** `opening_rate` (% of boards where this player opened the auction); `preempt_rate` (% of opening bids at level ≥ 2); `intervention_rate` (% of boards with a competitive bid into the opponents’ auction); `penalty_double_rate` (% of boards with a doubling call by this player); `avg_bids_per_board` (mean number of bids per board).

Note on legacy features. An earlier draft used a 5-feature set including `success_rate` and a composite `avg_risk_score`; both were dropped after the variance audit in Chapter 4 showed that `success_rate` provides almost no separating signal at elite level (all elite players succeed at $\approx 80\%$) and that `avg_risk_score` is a linear combination of other features and adds no new information.

After variance and correlation filtering (Chapter 4, Eq. (4.2)), the 10 features reduce to a stable 7–8 axis behaviour space depending on the configuration (8 for the production K-Means / extreme-percentile pipeline; 7 for the V3 audit configuration which also drops `partscore_rate`). Features are then standardised via `StandardScaler` (Eq. (4.1)), reduced via PCA (Eq. (4.4)), and assigned to one of five profiles via the extreme-percentile + binomial-test rule (Eq. (4.11)).

3.4 Stage 2 – Skill Extraction (LLM)

For each of the four extreme profiles we sampled five players and sent their actual played boards to **Gemini 2.5 Flash** in chunks of up to 25 boards, with a strict JSON schema. Each call returns 5–7 decision-making skills, each with a confidence label and supporting board references. A profile-specific guidance prompt (`PROFILE_GUIDANCE`) requires at least two skills to lie on the profile’s defining axis; without it, an unguided prompt returned the generic “aggressive competitive bidding” for every profile.

Semantic aggregation

Raw skill names vary in wording (e.g. “initiates slam exploration with splinters” vs. “employs splinter bids for slam exploration”). We merge synonymous skills with TF–IDF cosine similarity and Union–Find clustering (merge threshold 0.40; the lower value 0.30 produced false merges such as “control bidding” with “competitive overcalling”). The most frequent raw name in each cluster becomes the canonical skill label.

Empirical validation

The central question is whether the LLM-described profiles are *real* behavioural types or an artefact of the clustering / an LLM hallucination. We therefore measured each profile’s defining behaviour directly from the raw bidding and contract data—no LLM, no clustering—and compared it against a Generalist baseline drawn from the same elite population (an *elite-vs-elite* comparison, which makes any separation conservative). Denominators match Stage 1 exactly: declarer-only boards for the contract-level metrics (slam, partscore, NT) and per-board-with-bidding for the Fighter’s penalty doubles. Effect size is Cohen’s d ; significance is a one-sided Mann–Whitney U test. All four profiles separate from the baseline with large effect sizes and statistical significance (Table 3.2).

Table 3.2: Stage 2 empirical validation: each profile’s defining behaviour vs. the Generalist baseline (validation sample, five players per profile).

Profile	Defining metric	Ratio	Cohen’s d	p -value
Fighter	penalty_double_rate	$\times 1.31$	2.13	0.016
Insurance Player	partscore_rate	$\times 1.24$	3.30	0.004
Slam Hunter	slam_rate	$\times 1.37$	2.81	0.004
NT Specialist	nt_rate	$\times 1.27$	4.62	0.004

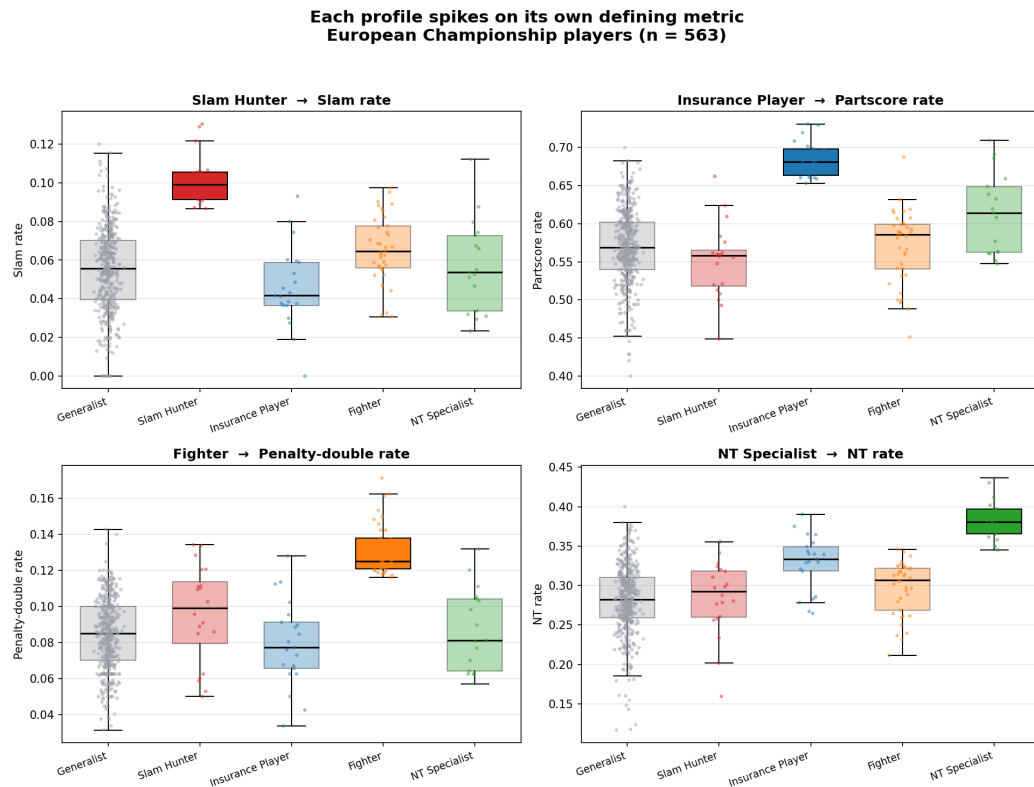
Cohen’s $d \geq 0.8$ is conventionally “large” and $d \geq 2.0$ “very large”; all four profiles clear $d = 2.0$ and all are significant at $p < 0.05$. (These ratios use the five-player validation sample and so differ slightly from the full-population ratios in Table A.2; both tell the same story.) A second independent review by a bridge-domain expert confirmed the result with the verdict “proceed to Stage 3”. The human-readable profile names are *Slam Hunter*, *Insurance Player*, *Fighter*, and *NT Specialist*.

Methodological note

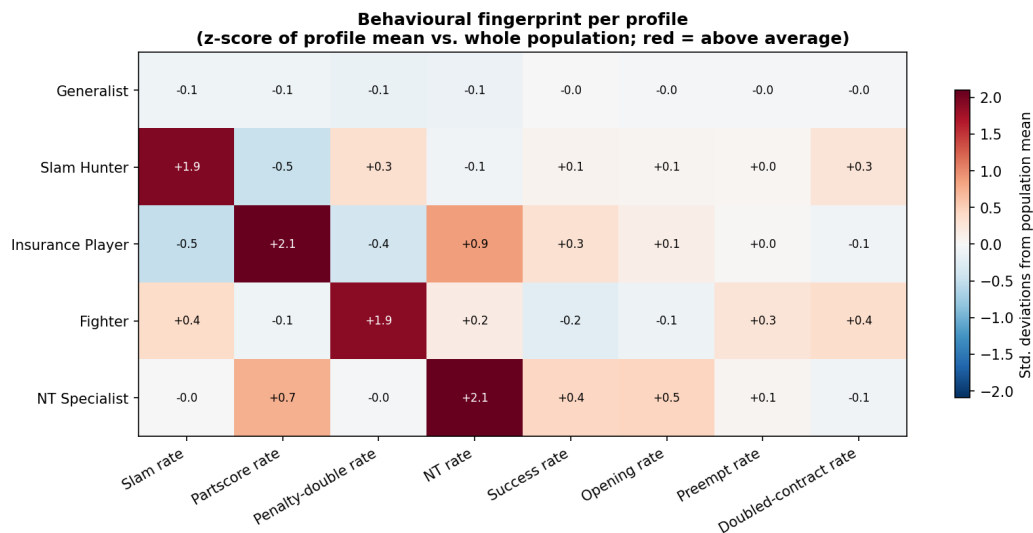
The Fighter metric was initially measured per call (doubles $\div$ total calls), which gave a weak, overlapping signal and was rejected on expert review. Switching to the Stage-1-aligned per-board metric (boards with at least one double $\div$ boards with bidding) produced the clean $\times 1.31$ separation above. The validation denominator must match the clustering denominator exactly, or a genuine profile can appear spurious.

Visual summary

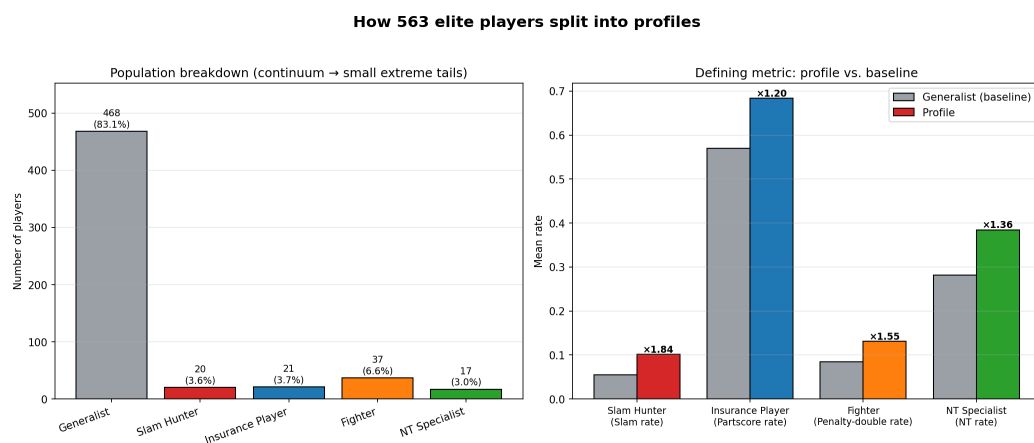
Figure 3.2 presents the validation visually: each profile spikes on its own defining metric (panel a), the per-profile “fingerprint” heatmap shows the selected axis plus coherent side-behaviours we did *not* select for (panel b), and the population splits into a dense Generalist continuum with the four profiles forming the behavioural tails (panel c).



(a) Each profile spikes on its defining metric.



(b) Per-profile behavioural fingerprint (z-scores vs. the population).



(c) Population breakdown and defining-metric ratios.

3.5 Stage 3 – Agent Construction

Each profile becomes an LLM agent through a structured system prompt (“character card”) that contains: identity (profile name), the 5–7 skills extracted in Stage 2, domain rules (bridge *or* negotiation), a strict JSON output schema, and one few-shot example. Crucially, the skills injected are the *bridge-derived* ones from Stage 2 (e.g. “Aggressive Penalty Doubling”), never hand-written personality words such as “be aggressive”. This is the anti-tautology safeguard: any cross-domain alignment measured in Stage 4 must *emerge* from bridge behaviour rather than from a personality label we supplied.

Architecture

A shared abstract `BaseAgent` holds the profile signature, the LLM client, and a single decision choke-point `_decide()` through which every call passes (centralising cost logging, JSON parsing, and retries). Two concrete agents inherit from it:

- `BridgeAgent.make_bid(hand, auction)` – returns a legal call (contract bid, `Pass`, `X`, or `XX`) plus one-line reasoning. A local Laws-of-Bridge legality check runs with no LLM; an illegal call is replaced by `Pass` (the guardrail against hallucinated bids).
- `NegotiationAgent.respond_to_offer(scenario, offer, history)` – returns an action (`counter/accept/walk_away`), a numeric offer clamped to the scenario’s legal range, a `willing_to_close` flag, and reasoning.

All construction is routed through the SDK (`build_bridge_agents()`, `build_nego_agents()`), honouring the single-entry-point methodology. Agents share one LLM client so cost is budget-capped across a whole game.

Reproducibility and the determinism finding

Bridge agents run at temperature 0 and negotiation agents at 0.7. A controlled test revealed that Gemini 2.5 Flash is **not deterministic even at temperature 0** (a known consequence of parallel GPU execution): the same hand can yield different bids across runs. The reproducibility strategy is therefore *sample-level*, not *call-level*: every raw output is persisted to JSON, and Stage 4 reports distributions over many hands/scenarios rather than single decisions. This is itself a methodological note for LLM-agent research.

Sanity validation

Two small real-LLM checks (saved under `experiments/`) confirm the wiring and that personality is visible before scaling up. On one 20-HCP hand the five agents produced three to four distinct calls, with the Slam Hunter driving toward slam and the Insurance Player signing off in game; on one M&A scenario the two safety/cooperative profiles (Insurance, Generalist) were the only ones willing to close while the aggressive/analytic profiles held out for a better price. A bridge-expert review confirmed the profile *directions* were correct and flagged bridge-mechanics errors (a splinter named on a doubleton, an under-strength 1NT overcall, opener/responder

seat confusion) and bridge-vocabulary leakage into the business reasoning. Both were fixed by adding convention-correctness, seat-awareness, and card-counting rules to the bridge card and a language constraint to the negotiation card; a re-run eliminated the flagged errors and the vocabulary leakage.

3.6 Stage 4 – Dual Simulation and Alignment

Stage 4 has been executed end-to-end; the numerical outcome is reported in Chapter 5. The design as run is as follows.

4a Bridge: 50 deterministically seeded deals are bid by each of the five profile agents sitting South as responder to a fixed sequence of partner openings; each deal is run three times and resolved by majority vote (Gemini is non-deterministic even at temperature 0). Each call is scored against the objective par-level contract implied by the partnership’s combined high-card strength.

4b Negotiation: four business scenarios (B2B SaaS pricing, co-founder equity split, customer-book acquisition, software-licence deal) are negotiated by each profile against a *fixed, non-LLM* seller that bargains by formula (concedes only toward a floor), for up to four rounds with an explicit walk-away condition. The metric is the surplus captured, (seller open – deal price) / (seller open – seller floor); each scenario is run three times.

4c Alignment: *winning* ↔ *winning* alignment is weak and metric-sensitive ($\rho \approx +0.2$ to $+0.5$), because whether a style wins depends on the opponent. Refining to the behavioural form – *style* ↔ *style* (bridge aggression vs negotiation aggression) – gives $\rho = +0.80$, above the 0.70 target; an inverse-prompt control (swap each profile’s skills) flips it to -0.90 , proving the transfer is skill-mediated, not tautological. A random “monkey” baseline and a double-dummy evaluator validated the bridge metric. Full details and figures are in Chapter 5.

3.7 Tools and Stack

Development environment

- **Language:** Python 3.11 in an isolated `venv` virtual environment.
- **Operating system:** Windows 11 with WSL2 for Unix-style tooling.
- **Primary IDE:** Google Antigravity (agent-first development).
- **Secondary IDE:** Microsoft Visual Studio Code.
- **AI coding assistant:** Anthropic Claude Code (CLI), with custom slash commands (e.g. `/bridge-expert`) defined in `.claude/commands/`.
- **Source control:** Git + GitHub, public repository.

Data science and statistical libraries

- **Data manipulation:** pandas, numpy.

- **Machine learning (scikit-learn):** `StandardScaler`, `RobustScaler` for Z-score normalisation; `PCA` for dimensionality reduction; `KMeans`, `HDBSCAN`, `GaussianMixture` for clustering; `TSNE` for visualisation; `EmpiricalCovariance` for Mahalanobis-distance outlier detection.
- **Statistics:** `scipy.stats.binomtest` (one-sided binomial test for profile assignment); `scipy.stats.chi2` (Mahalanobis threshold).
- **Future stages:** `gensim` (Word2Vec for bidding sequences), `shap` (XAI for cluster explanation).

Visualisation and reporting

- **Plots:** `matplotlib` – PCA scatter, t-SNE scatter, radar charts, scree plots, feature bar charts.
- **Spreadsheets:** `openpyxl` – transparent export of normalised features and PCA loadings to `results/preprocessing.xlsx`.
- **This report:** $\text{\LaTeX}$ (TeX Live), `natbib`, `booktabs`, `tabularx`.
- **Internal documentation:** Markdown for PRD, `RESEARCH_INSIGHTS`, `CLAUDE.md`, and `README` files.

LLM providers (provider-neutral architecture)

- **Default:** Google Gemini Flash 2.0 – cost \$0.075 / M input, \$0.30 / M output tokens.
- **Cross-validation:** Anthropic Claude (Haiku / Opus) and OpenAI GPT, routed through a single `shared/llm_client.py` wrapper. Running the same prompt across providers strengthens the claim that findings are not a single-provider artefact.
- **Configuration:** all API keys loaded from `.env` via `python-dotenv`; never hard-coded.
- **Retry handling:** `tenacity` for exponential back-off under rate limits.

Testing and quality

- **Testing:** `pytest` (73 unit tests passing across feature engineering, profile assignment, and the bridge-expert validator); `pytest-cov` for coverage reporting.
- **Linting and formatting:** `ruff`.
- **Type validation:** `pydantic` for structured LLM outputs.

Experiment tracking and reproducibility

- **Per-call logs:** JSON-Lines records in `results/llm_logs/` with provider, model, prompt tokens, completion tokens, latency, and cost.
- **Random seeds:** fixed `random_state=42` across scikit-learn calls for reproducible clustering and PCA.

- **Pipeline replay:** the entire Stage 1 pipeline runs end-to-end from a clean clone with a single command via `src/sdk.py`.

Source data

- **EuroBridge public database** – five European Championships (2016–2025).
- **Custom Python scraper** – `requests` + `BeautifulSoup4`, developed in the parent `collect-BridgeData` project.

4 Technical Architecture and Mathematical Foundations

This chapter formalises the algorithms and architectures used by NegoPlay. It is written for technical reproducibility: any reader with the published dataset and the equations below should be able to reproduce every numerical result in this report.

4.1 System Architecture

NegoPlay is a four-stage hybrid pipeline (ML $\rightarrow$ LLM $\rightarrow$ LLM $\rightarrow$ LLM) connected by file artefacts on disk. Figure 4.1 shows the system architecture with stages, intermediate artefacts, and the direction of information flow.

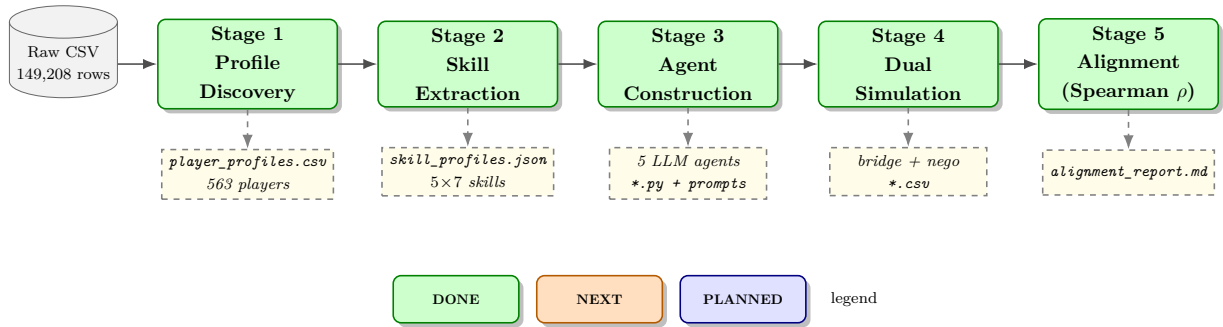


Figure 4.1: NegoPlay system architecture. Solid arrows: information flow between stages. Dashed arrows: persisted artefacts on disk. All five stages are complete: the full pipeline ran end-to-end from the 149K-row raw dataset to the cross-domain alignment result (Chapter 5).

4.2 Stage 1: Mathematical Pipeline

Stage 1 transforms the raw EuroBridge CSV into a labelled profile assignment for each qualifying player. Figure 4.2 shows the full preprocessing and assignment pipeline.

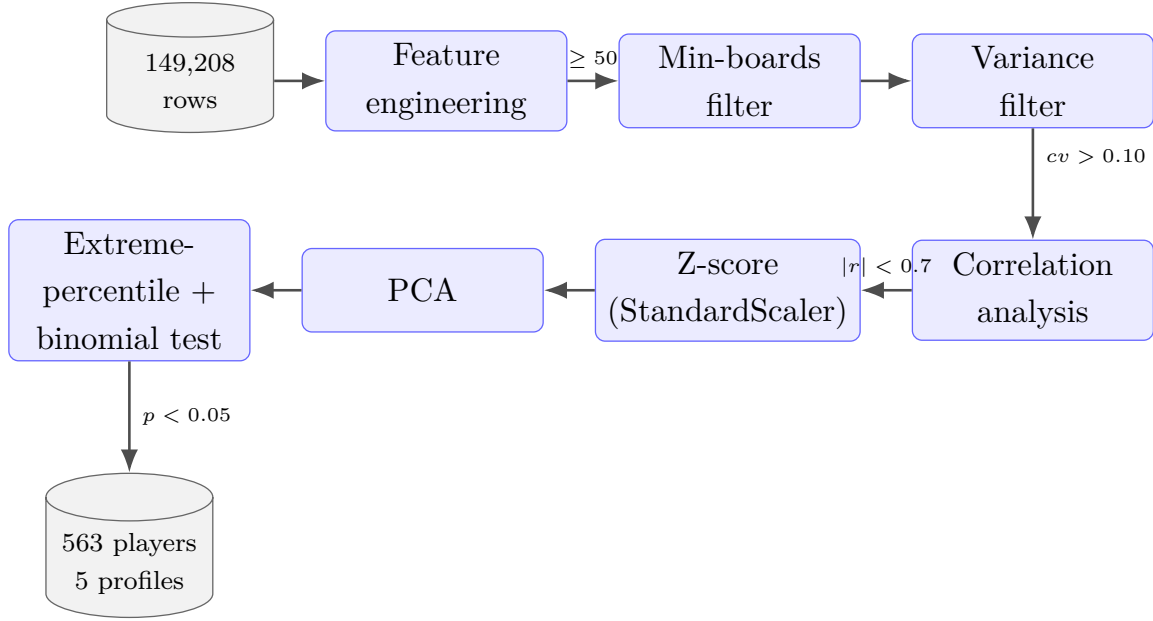


Figure 4.2: Stage 1 pipeline: raw data → feature engineering → statistical filters → scaling → PCA → extreme-percentile profile assignment. Each filter is labelled with its threshold.

4.2.1 Notation

Let $\mathbf{X} \in \mathbb{R}^{n \times p}$ denote the player-feature matrix where $n = 563$ is the number of qualifying players (those with ≥ 50 declared boards *and* ≥ 50 bidding boards). The i -th row $\mathbf{x}_i$ represents one player; the j -th column $\mathbf{X}_{.j}$ represents one feature.

Two configurations of p . The feature count depends on the pipeline:

- **Production pipeline ($p = 8$):** we start from the 10 features of Section 3, then drop `avg_level` ($CV = 0.04$) and `avg_bids_per_board` ($CV = 0.08$) for failing the variance gate of Eq. (4.2).
- **V3 audit configuration ($p = 7$):** same as production, but `partscore_rate` ($CV = 0.09$) is also dropped as borderline-low-variance during a preprocessing audit. The Mahalanobis-distance outlier detection below is run in the V3 configuration, hence $p = 7$ in Eq. (4.10).

4.2.2 Z-score standardisation

To make features comparable across different scales (e.g., `slam_rate` $\in [0, 0.15]$ vs. `nt_rate` $\in [0, 0.45]$), each feature is standardised to zero mean and unit variance:

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j} \quad (4.1)$$

Parameters:

z_{ij} standardised score of player i on feature j .

x_{ij} raw value of feature j for player i (e.g., player’s observed `slam_rate`).

μ_j population mean of feature j across all $n = 563$ qualifying players: $\mu_j = \frac{1}{n} \sum_{i=1}^n x_{ij}$.

σ_j population standard deviation of feature j : $\sigma_j = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_{ij} - \mu_j)^2}$.

Equation (4.1) is implemented by `sklearn.preprocessing.StandardScaler`. A z -score of $+2$ means the player is two population standard deviations above the mean on that feature; in a normal distribution this corresponds to the $\approx 97.5^{\text{th}}$ percentile. We use the same formula both for preprocessing (input to K-Means / PCA) and for profile-significance scoring (column `profile_z` in `player_profiles.csv`).

4.2.3 Variance audit (coefficient of variation)

Following a preprocessing review, we audit each feature by its coefficient of variation:

$$\text{CV}_j = \frac{\sigma_j}{|\mu_j|} \quad (4.2)$$

Parameters:

CV_j coefficient of variation of feature j (dimensionless).

σ_j population standard deviation of feature j (as in Eq. (4.1)).

μ_j population mean of feature j (as in Eq. (4.1)).

Features with $\text{CV}_j < 0.10$ are discarded as low-information: their relative spread is below 10% of the mean, so they cannot meaningfully distinguish players. In our data, `avg_level` ($\text{CV} = 0.04$) and `avg_bids_per_board` ($\text{CV} = 0.08$) were removed by this filter.

4.2.4 Principal Component Analysis

PCA computes the eigendecomposition of the standardised covariance matrix:

$$\mathbf{C} = \frac{1}{n-1} \mathbf{Z}^\top \mathbf{Z} \quad (4.3)$$

Parameters:

$\mathbf{C}$ $p \times p$ covariance matrix of the standardised features.

$\mathbf{Z}$ $n \times p$ matrix of z -scores from Eq. (4.1) ($n = 563$, $p = 8$).

$n-1$ degrees-of-freedom correction (Bessel's correction).

Solving the eigenvalue problem

$$\mathbf{C} \mathbf{v}_k = \lambda_k \mathbf{v}_k, \quad k = 1, \dots, p \quad (4.4)$$

Parameters:

$\mathbf{v}_k$ k -th eigenvector (principal direction in feature space).

λ_k k -th eigenvalue, ordered $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p$.

k component index, $k \in \{1, \dots, p\}$.

The fraction of variance explained by component k is

$$\text{EV}_k = \frac{\lambda_k}{\sum_{i=1}^p \lambda_i} \quad (4.5)$$

We retain the top K components that satisfy $\sum_{k=1}^K \text{EV}_k \geq 0.80$ (cumulative variance threshold). In our data, $K = 5$ components achieve 85.2% cumulative variance: $\text{EV} = (0.246, 0.191, 0.135, 0.103, 0.077)$.

4.2.5 Clustering validity: Silhouette score

For each player i assigned to cluster $C_{k(i)}$, define:

$$a(i) = \frac{1}{|C_{k(i)}| - 1} \sum_{\substack{j \in C_{k(i)} \\ j \neq i}} d(\mathbf{x}_i, \mathbf{x}_j) \quad (4.6)$$

$$b(i) = \min_{\ell \neq k(i)} \frac{1}{|C_\ell|} \sum_{j \in C_\ell} d(\mathbf{x}_i, \mathbf{x}_j) \quad (4.7)$$

Parameters:

$a(i)$ mean Euclidean distance from player i to all *other* players in their own cluster $C_{k(i)}$ – the “cohesion” term.

$b(i)$ mean Euclidean distance from player i to the nearest *other* cluster – the “separation” term. The min ranges over all clusters $\ell \neq k(i)$.

$d(\mathbf{x}_i, \mathbf{x}_j)$ Euclidean distance in the scaled PCA space.

$C_{k(i)}$ the cluster to which player i is assigned.

The silhouette score for player i and the overall mean are:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \quad \bar{s} = \frac{1}{n} \sum_{i=1}^n s(i) \quad (4.8)$$

Range: $s(i) \in [-1, +1]$ for every player. Interpretation:

$\bar{s} \geq 0.50$ strong cluster structure.

$0.25 \leq \bar{s} < 0.50$ weak structure.

$\bar{s} < 0.25$ no meaningful cluster structure.

Our maximum across five preprocessing configurations was $\bar{s} = 0.24$ (V2: 8 features + PCA(3), Z-score). Computed via `sklearn.metrics.silhouette_score`.

4.2.6 Mahalanobis-distance outlier detection

For multivariate outlier detection prior to clustering (V3 configuration only), we use the squared Mahalanobis distance:

$$D^2(\mathbf{x}_i) = (\mathbf{x}_i - \boldsymbol{\mu})^\top \mathbf{S}^{-1} (\mathbf{x}_i - \boldsymbol{\mu}) \quad (4.9)$$

Parameters:

$D^2(\mathbf{x}_i)$ squared Mahalanobis distance of player i from the data centre.

$\mathbf{x}_i$ feature vector of player i (after scaling).

$\boldsymbol{\mu}$ sample mean vector of all players, $\boldsymbol{\mu} \in \mathbb{R}^p$.

$\mathbf{S}^{-1}$ inverse of the empirical covariance matrix (estimated by `sklearn.covariance.EmpiricalCovariance`)

The inverse weights features by their inter-correlations, so a player who is unusual on *correlated* features counts as more outlying.

Under multivariate normality, $D^2 \sim \chi_p^2$, so player i is flagged as an outlier when

$$D^2(\mathbf{x}_i) > \chi_{p, 1-\alpha}^2 \quad (4.10)$$

Parameters:

$\chi_{p, 1-\alpha}^2$ $(1 - \alpha)$ critical value of the chi-square distribution with p degrees of freedom (number of features). Computed via `scipy.stats.chi2.ppf(1- $\alpha$ , df= $p$ )`.

α significance level; we use $\alpha = 0.01$ (top 1% of the tail). For $p = 7$, this gives a threshold of ≈ 18.5 .

Methodological note: outlier removal is appropriate for K-Means and GMM but counterproductive for the extreme-percentile method, since the outliers are precisely the profile members we want to identify.

4.2.7 Extreme-percentile profile assignment

The production pipeline uses an extreme-percentile method on the standardised features, supported by a binomial significance test. For each profile axis j (`slam_rate`, `partscore_rate`, `nt_rate`, `penalty_double_rate`), we:

1. Identify the top 10% of players on z_j (Eq. 4.1).
2. Test the null hypothesis that their observed rate equals the population baseline p_0 using a one-sided binomial test (Eq. 4.11).
3. Assign the player to profile j only if both gates pass.

$$\text{p-value}_i = \sum_{m=k_i}^{n_i} \binom{n_i}{m} p_0^m (1 - p_0)^{n_i - m} \quad (4.11)$$

Parameters:

n_i player i 's denominator: `n_declared` for outcome features (Slam Hunter, Insurance Player, NT Specialist) or `n_bidding_boards` for process features (Fighter).

k_i number of “successes” for player i on the profile axis (e.g., number of slam contracts declared, for Slam Hunter).

p_0 population baseline rate of the event (e.g., $p_0 = 0.055$ for slam).

$\binom{n_i}{m}$ binomial coefficient “ n_i choose m ”.

m summation index over all outcomes at least as extreme as the observed k_i .

Interpretation: the p-value is the probability of observing at least k_i successes under the null hypothesis that the player's true rate equals the baseline p_0 (one-sided, “greater than” alternative). We require $\text{p-value}_i < 0.05$ for profile assignment. Implemented as `scipy.stats.binomtest( $k_i$ ,  $n_i$ ,  $p_0$ , alternative='greater')`.

4.3 Stage 3: LLM Agent Architecture

Each profile becomes an LLM agent through a structured prompt. Figure 4.3 shows the agent’s request/response cycle.

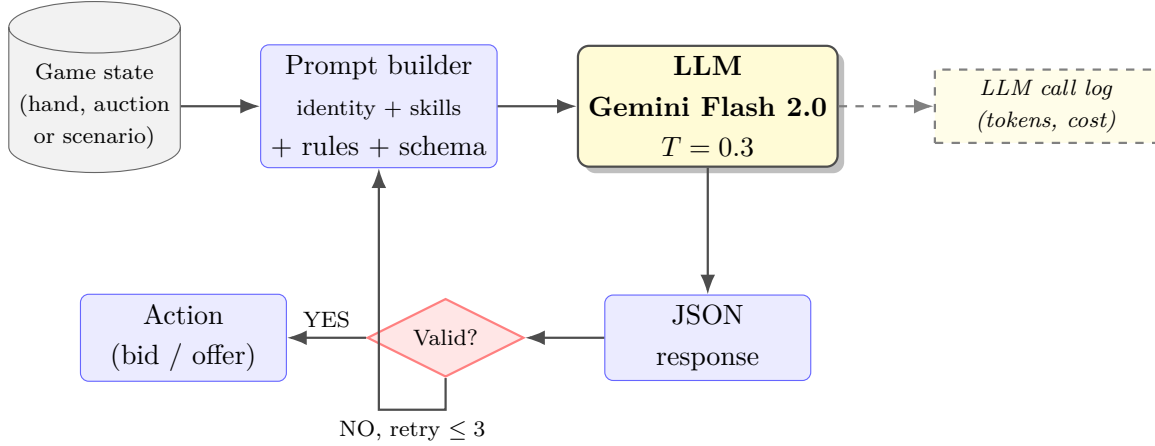


Figure 4.3: LLM agent architecture: game state → prompt builder → Gemini → JSON response → validator → action. Invalid responses trigger a retry with the validation error in the next turn. Every call is logged with token counts and cost for reproducibility.

4.3.1 System prompt anatomy

Every agent’s system prompt has the same five-block structure. We formalise this so that the four profile agents differ only in the content of blocks 1 and 2:

Listing 4.1: Five-block system prompt template. Filled blocks become the production prompt in `shared/prompts.py`.

```

1 [BLOCK 1 - IDENTITY]
2 You are a <profile_name>. <one-line role description>.
3
4 [BLOCK 2 - CORE SKILLS]
5 Your 5-7 characteristic skills (extracted from real bridge hands):
6   - <skill 1>
7   - <skill 2>
8   ...
9
10 [BLOCK 3 - DECISION RULES]
11 Domain rules (Laws of Duplicate Bridge / negotiation conventions).
12
13 [BLOCK 4 - OUTPUT FORMAT]
14 Respond ONLY with this JSON schema:
15 { "decision": "...", "reasoning": "...", "confidence": 0.0-1.0 }
16
17 [BLOCK 5 - FEW-SHOT EXAMPLES]
18 Example 1: <hand> -> <bid> (reasoning: ...)
19 Example 2: <scenario> -> <offer> (reasoning: ...)
  
```

4.3.2 Temperature sampling

LLM token selection follows a softmax over logits, modulated by a temperature parameter T :

$$P(\text{token} = t \mid \text{context}) = \frac{\exp(z_t/T)}{\sum_{t'=1}^V \exp(z_{t'}/T)} \quad (4.12)$$

Parameters:

$P(t \mid \text{ctx})$ probability that the model emits token t next, given context.

z_t raw logit of token t produced by the model’s final linear layer.

V vocabulary size (Gemini’s tokeniser: $\approx 256\text{K}$ tokens).

T temperature parameter, $T > 0$. As $T \rightarrow 0$ the distribution becomes peaked (deterministic, argmax); as $T \rightarrow \infty$ it becomes uniform.

Choices for NegoPlay:

- $T = 0.3$ for bridge bidding (consistency is critical: the same hand should produce the same bid).
- $T = 0.7$ for negotiation (diversity of offers is part of the experiment).
- $T = 0.1$ for the Bridge-Expert Validation skill (we want deterministic expert opinions).

4.3.3 Per-call cost accounting

Each LLM call writes a JSON-Lines record to `results/llm_logs/`. The cost is computed as

$$C_{\text{call}} = \frac{T_{\text{in}} \cdot c_{\text{in}} + T_{\text{out}} \cdot c_{\text{out}}}{10^6} \quad (4.13)$$

Parameters:

C_{call} cost of a single LLM call, in US dollars.

T_{in} number of input tokens (system prompt + user message), reported by the provider’s API metadata.

T_{out} number of output tokens (completion).

c_{in} unit cost per 10^6 input tokens (\$0.075 for Gemini Flash 2.0).

c_{out} unit cost per 10^6 output tokens (\$0.30 for Gemini Flash 2.0).

The cumulative cost is monitored against a hard cap of \$50, with an alert at \$20.

4.4 Stage 4: Dual Simulation and Alignment Measure

The core empirical claim of NegoPlay is measured by a single number: the Spearman rank correlation between profile win-rates in bridge and in negotiation.

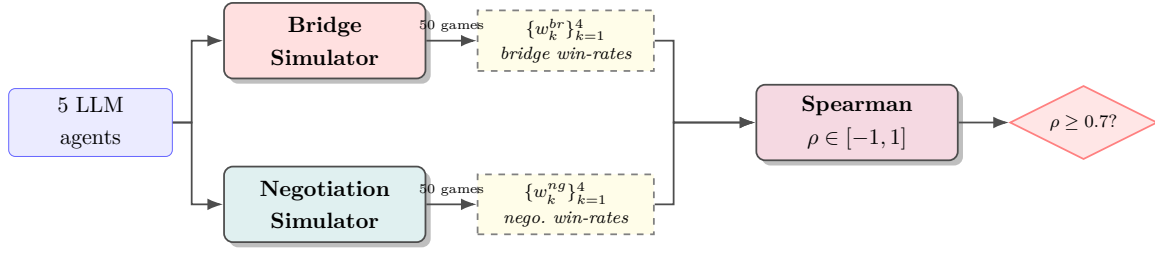


Figure 4.4: Stage 4 dual simulation: the same agents play bridge and negotiation, producing two vectors of profile-level win-rates. Spearman correlation between them is the main outcome.

4.4.1 Per-profile win rate

$$w_k^D = \frac{1}{N_D} \sum_{g=1}^{N_D} \mathbf{1}[\text{agent}_{P_k} \text{ won game } g] \quad (4.14)$$

Parameters:

w_k^D win-rate of profile k in domain $D \in \{\text{bridge}, \text{negotiation}\}$, $w_k^D \in [0, 1]$.

N_D number of games in domain D ($N_{\text{bridge}} = N_{\text{negotiation}} = 50$ in Phase 1).

$\mathbf{1}[\cdot]$ indicator function: 1 if the inner condition is true, 0 otherwise.

P_k one of the four behavioural profiles ($k \in \{1, \dots, 4\}$), excluding the Generalist baseline.

4.4.2 Spearman rank correlation

The main outcome measure of NegoPlay. Let r_k^{br} and r_k^{ng} be the ranks of profile k within its domain (rank 1 = highest win-rate). The Spearman correlation, when there are no ties, is

$$\rho_s = 1 - \frac{6 \sum_{k=1}^K d_k^2}{K(K^2 - 1)}, \quad d_k = r_k^{br} - r_k^{ng} \quad (4.15)$$

Parameters:

ρ_s Spearman rank correlation, $\rho_s \in [-1, +1]$. Value +1 means perfect rank agreement; -1 means perfect rank inversion; 0 means no correlation.

K number of profiles compared ($K = 4$).

d_k difference in ranks of profile k between bridge and negotiation.

The general formula (which handles ties correctly) is the Pearson correlation applied to the ranks:

$$\rho_s = \frac{\sum_k (r_k^{br} - \bar{r}^{br})(r_k^{ng} - \bar{r}^{ng})}{\sqrt{\sum_k (r_k^{br} - \bar{r}^{br})^2 \sum_k (r_k^{ng} - \bar{r}^{ng})^2}} \quad (4.16)$$

Parameters:

r_k^{br}, r_k^{ng} ranks of profile k in bridge and negotiation respectively.

$\bar{r}^{br}, \bar{r}^{ng}$ mean ranks (always $(K + 1)/2 = 2.5$ for $K = 4$).

Decision rule: $\rho_s \geq 0.70 \Rightarrow$ support for H3 (alignment hypothesis); $0.50 \leq \rho_s < 0.70 \Rightarrow$ partial support; $\rho_s < 0.50 \Rightarrow$ null result. Significance is tested by a two-sided permutation

test with 10,000 random permutations of the negotiation win-rate vector. Implemented as `scipy.stats.spearmanr`.

4.4.3 Falsifiability test (Phase 3)

To rule out the *tautological alignment* confound (*i.e.*, the LLM merely following the prompt rather than transferring a learned style), we run an inverse-prompt control:

$$\rho_{\text{inverse}} = \text{Spearman}(\{w_k^{br}\}_{k=1}^K, \{w_k^{ng, \text{inverse}}\}_{k=1}^K) \quad (4.17)$$

Parameters:

$w_k^{ng, \text{inverse}}$ negotiation win-rate of an agent prompted with the *opposite* profile (e.g., a Slam Hunter in bridge becomes an Insurance-style agent in negotiation).

ρ_{inverse} alignment under the inverted-prompt control.

Falsifiability criterion: if $\rho_{\text{inverse}} \geq 0.70$, the original alignment is a prompt-following artefact, not a true behavioural transfer, and H3 must be rejected even when the main ρ_s is above threshold.

Executed (June 2026): the control was run on the style measure and gave $\rho_{\text{inverse}} = -0.90$ – far below the artefact threshold, *i.e.* the test *passed* (see Chapter 5).

5 Results: Stage 4 Cross-Domain Alignment

This chapter reports the executed Stage 4 outcome, and it is honest about a refinement made along the way. We first asked the *winning* form of the question – does a profile that *wins* at bridge also *win* at negotiation? – built a rigorous bridge metric, and found the answer *weak and metric-sensitive*. We then realised that this caught the wrong quantity: whether a style wins depends on the opponent and the payoff rules, whereas the original hypothesis is about *behavioral alignment* – i.e. whether the decision *style* transfers. Refining the question from *winning*↔*winning* to *style*↔*style* gives a strong, controlled result ($\rho = +0.80$). We report both, in that order.

5.1 Step 1 (winning): building a trustworthy bridge metric

To score bridge success we added (i) a **double-dummy evaluator** – the bridge-standard objective reference, which computes the optimal result of a contract from all 52 cards – and (ii) a **random “monkey” baseline**, an agent that bids randomly but legally. A valid skill metric must rank the monkey last; if it does not, it is not measuring skill.

The monkey was decisive. On a coarse “did you bid roughly the right level” proxy, the random monkey *outscored* every expert profile (0.54 versus 0.25–0.51), proving that proxy meaningless. Re-scored by whether each contract *actually makes* under double-dummy play, the monkey collapses to last (0.10) and every profile beats it (Figure 5.1).

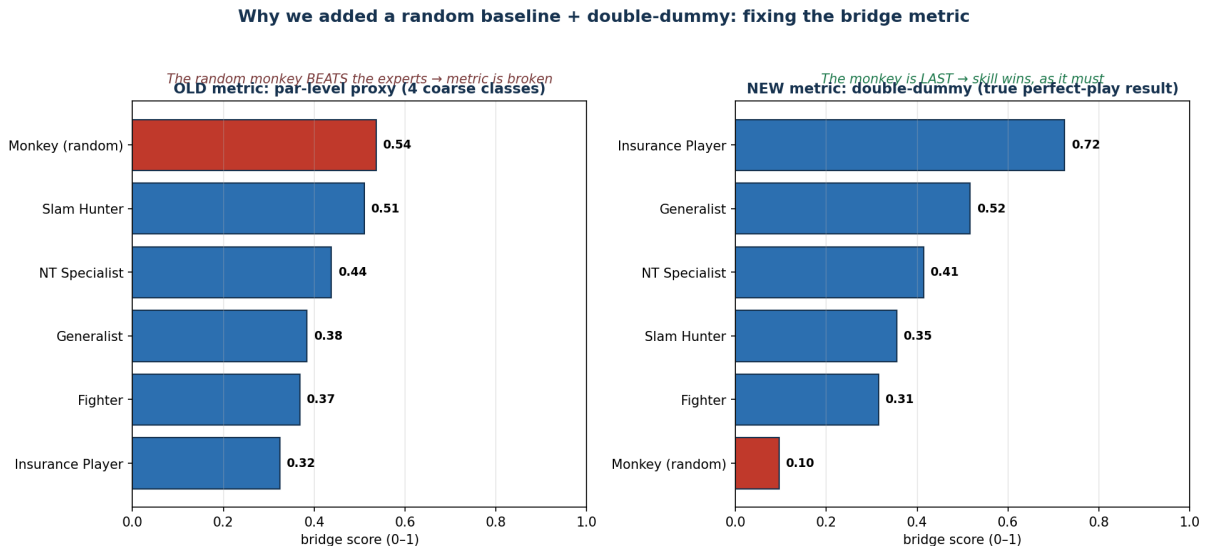


Figure 5.1: A random baseline exposes, and a double-dummy metric fixes, the bridge score. *Left:* on the coarse proxy the random monkey (red) beats every profile. *Right:* scored by whether the contract actually makes (double-dummy), the monkey is last and skill wins.

5.2 Measuring bridge from the real competitive data

Rather than simulate bridge, we measured each profile’s *real* competitive performance directly from the EuroBridge data. In duplicate teams every board is played at two tables, so the same North–South problem has two results; the board *datum* is their average, and a pair’s result on that board is its score minus the datum (positive = better than the other table on identical cards). We attribute that to the four seated players, average per player, then per profile, and express it in **IMPs** (the standard bridge unit, which compresses large boards so no single swing dominates). Crucially this counts *defence* automatically – a successful penalty double scores positive for the defending side – so the Fighter’s signature skill is captured with *no* hand-tuned weight.

Table 5.1: Real competitive bridge skill (IMP versus the field, defence included) and the simulated negotiation surplus, per profile.

Profile	Bridge IMP (primary)	Bridge points (robustness)	Negotiation
Slam Hunter	+0.172	+4.08	0.443
Fighter	+0.085	+2.05	0.444
Generalist	+0.008	+0.19	0.362
Insurance Player	−0.006	−0.63	0.386
NT Specialist	−0.010	−0.24	0.400

5.3 Winning alignment: weak and metric-sensitive

The cross-domain Spearman correlation between real bridge skill and negotiation surplus is $\rho_s = +0.50$ on the IMP scale and $+0.60$ on raw points (Figure 5.2). But this *winning*-based number is fragile: across reasonable choices of bridge metric and of the negotiation seller it ranged from $+0.20$ to $+0.80$ and even flipped to $−0.90$ under an over-harsh walk-away seller. The reason is structural – whether a style *wins* depends on the opponent and the payoff rules – so winning $\leftrightarrow$ winning is too noisy to carry the claim. This motivates Step 2.

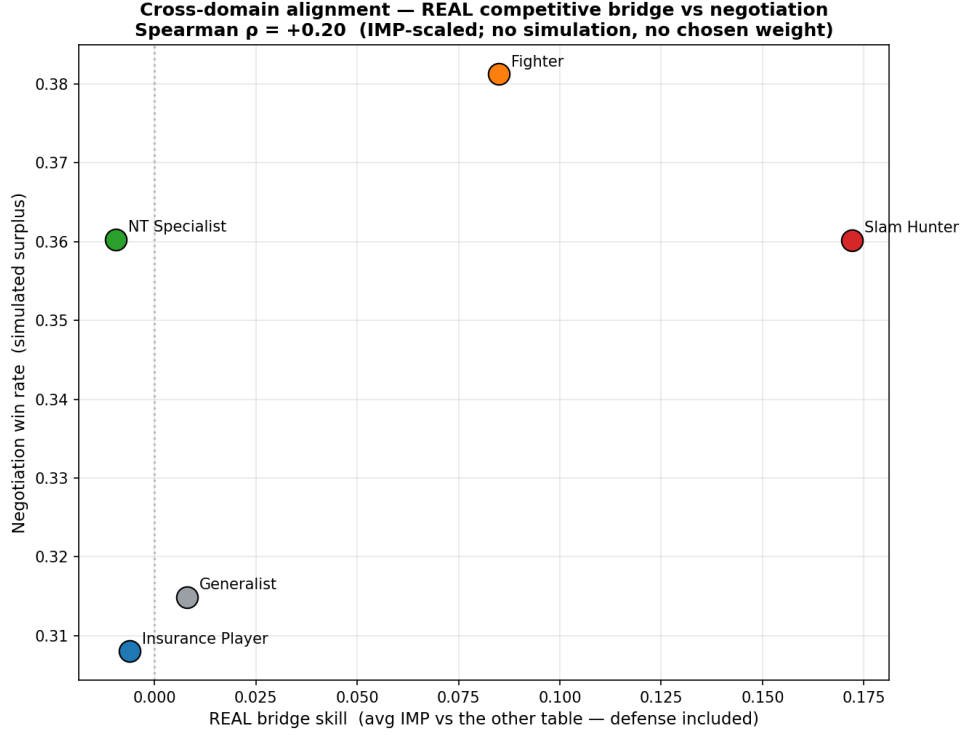


Figure 5.2: Real competitive bridge skill (IMP, x) versus simulated negotiation surplus (y). Spearman $\rho = +0.50$; the aggressive profiles lead in both domains.

5.4 The skill spectrum: random floor to perfect ceiling

To place every agent between a floor and a ceiling on the *real* boards, we scored a random monkey (a random contract, double-dummy scored from the 52 cards) and double-dummy par (perfect play), both in IMP versus the field (Figure 5.3). The random monkey sits at -7 IMP (a disaster); the real elite players cluster near 0 (all skilled); double-dummy perfect play is $+1.1$ IMP (even elite players leave ~ 1 IMP on the table). Among the profiles, the aggressive ones lead – consistent with the result above.

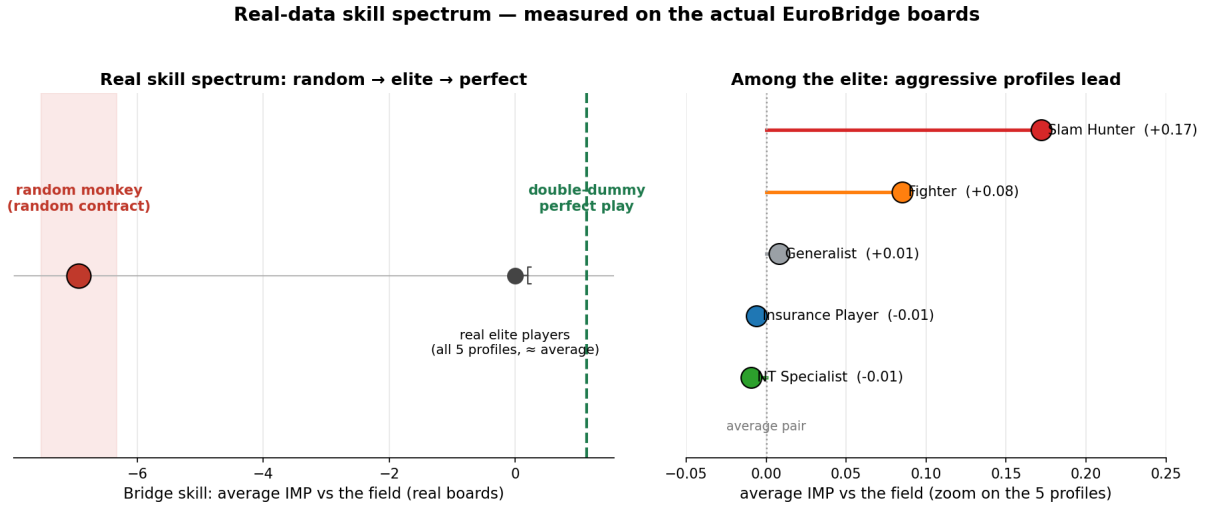


Figure 5.3: Real-data skill spectrum on the actual boards. *Left*: random monkey (−7 IMP) → elite players (≈ 0) → double-dummy perfect (+1.1 IMP). *Right (zoom)*: among the elite, the aggressive profiles (Slam Hunter, Fighter) lead.

5.5 Step 2 (style): does the decision STYLE transfer?

We refine the question to the behavioural form. **Bridge aggression** is a composite z-score of the real aggressive bidding features (slam, preempt, penalty-double rates). **Negotiation aggression** is how low the profile’s agent opens (the “demand depth” of its offers, from the saved logs). Correlating the two across the five profiles gives **Spearman** $\rho = +0.80$ ($p = 0.10$): the aggressive profiles (Fighter, Slam Hunter) bargain aggressively, the cautious ones (Insurance, Generalist) bargain softly (Figure 5.4, left). This clears the pre-registered $\rho \geq 0.70$ behavioral-alignment target.

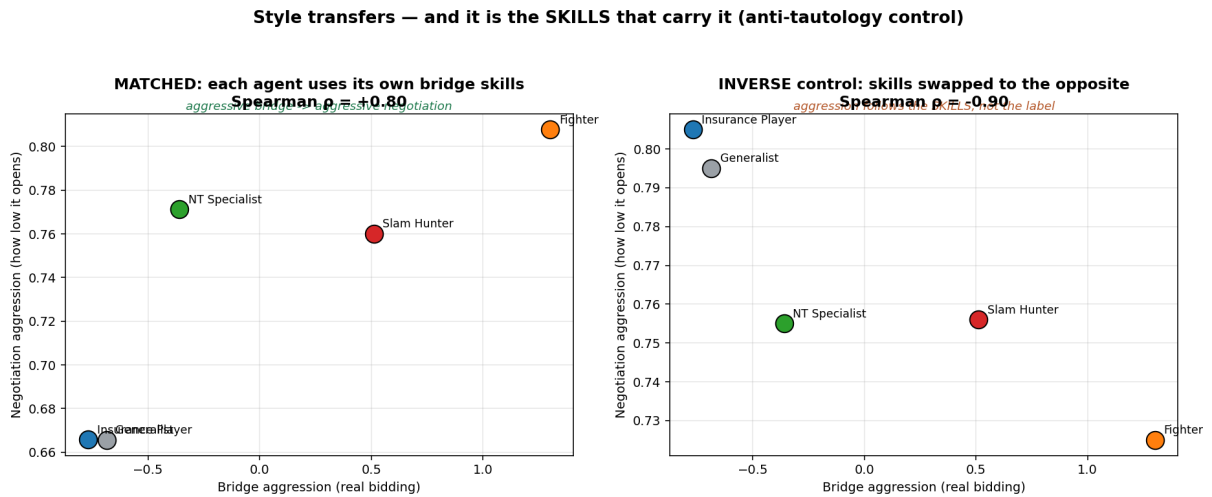


Figure 5.4: *Left (matched)*: bridge aggression vs negotiation aggression, $\rho = +0.80$ – the style transfers. *Right (inverse control)*: when each profile is given the *opposite* profile’s bridge skills, the correlation flips to $\rho = -0.90$ – negotiation aggression follows the injected skills, not the label.

5.6 Anti-tautology control

A natural objection: the agent may bargain aggressively only because it was *labelled* an aggressive profile. We test this with an inverse-prompt control: keep each profile’s identity but inject the *opposite* profile’s bridge skills, then re-measure. If behaviour follows the (bridge-derived) skills, the correlation should flip. It does – from $+0.80$ to $\rho = -0.90$ ($p = 0.037$): the Fighter given the Insurance Player’s skills bargains *less* aggressively, the Insurance Player given the Fighter’s skills bargains *more* aggressively (Figure 5.4, right). The style is therefore carried by the *skills extracted from real bridge play*, not by the profile name – so the $+0.80$ alignment is **not** tautological. The negotiation behaviour and the seller’s mild walk-away were additionally calibrated against 5,247 real Craigslist negotiations.

5.7 Interpretation and caveats

- **Style transfers; winning is noisy.** The decision *style* transfers strongly ($\rho = +0.80$, controlled), while *winning* transfers weakly ($\rho \approx +0.2$). This is a substantive distinction, not a failure: an aggressive bridge player *behaves* aggressively in negotiation, but whether that wins depends on the domain’s payoff rules.
- **The behavioral-alignment hypothesis is supported**, and the anti-tautology control shows the effect is mediated by bridge-derived skills rather than labels.
- **Honest caveats.** With $n = 5$ profiles, power is low (p not significant); ρ is an *indication*. The Slam Hunter profile, defined by *bidding* slams, may partly proxy overall strength. Negotiation is simulated – no real same-person negotiation data exists; a scaled inverse-prompt control and real negotiation data are Year-2 directions.

6 Schedule

The project runs for five weeks (ten sessions, two per week). Each week corresponds to a sprint with one major deliverable.

Table 6.1: Five-week sprint plan.

Sprint	Week	Theme	Deliverable
1	1	Research & Setup	PRD, literature review, repo skeleton
2	2	Stage 1 + Stage 2	Validated clusters, named profiles
3	3	Stage 3	Four working LLM agents
4	4	Stage 4	Bridge + negotiation simulations + alignment
5	5	Polish & Defense	Final report, presentation, demo video

7 Work Packages

The project is decomposed into seven work packages (WP). Each WP groups related tasks across one or more sessions.

WP1 – Research foundation

Sessions: 1–2. **Deliverables:** PRD v1.0, literature review (17 papers + 9 PhD-context), repository structure, two anchor papers identified.

Tasks: (T1.1) project scope alignment with supervisor; (T1.2) PRD draft and freeze; (T1.3) literature search and anchor selection; (T1.4) GitHub repository scaffold; (T1.5) Antigravity + Gemini setup; (T1.6) related-work survey of eight bridge-AI repositories.

WP2 – Data pipeline (already in place at project start)

Sessions: pre-project (Jan–May 2026). **Deliverable:** `all_matches_full.csv` – 149,208 rows.

Tasks: EuroBridge HTML scraping; card backfill via BoardAcross; player-name backfill (99.9 % coverage). *This WP is closed; the report documents it but no further work is required for Nego-Play.*

WP3 – Stage 1: clustering

Sessions: 3–4. **Deliverables:** `player_features.csv`, `player_clusters.csv`, silhouette plot, validation report.

Tasks: (T3.1) `shared/data_loader.py`; (T3.2) `stage1_clustering/features.py`; (T3.3) `stage1_clustering/clustering.py` (K-Means + HDBSCAN); (T3.4) `stage1_clustering/validation.py` (Silhouette, Davies-Bouldin, permutation p -value); (T3.5) exploratory notebook with PCA / t-SNE plots.

WP4 – Stage 2: skill extraction

Sessions: 4. **Deliverables:** `skill_profiles.json`, `docs/profile_descriptions.md`.

Tasks: (T4.1) `shared/llm_client.py` (unified Gemini/Claude/OpenAI wrapper); (T4.2) game chunker; (T4.3) Gemini-based skill extractor with strict JSON schema; (T4.4) cross-chunk aggregator; (T4.5) manual profile naming by the author.

WP5 – Stage 3: agents

Sessions: 5–6. **Deliverables:** `BridgeAgent`, `NegotiationAgent`, full prompt book.

Tasks: (T5.1) `BaseAgent` abstract class; (T5.2) `BridgeAgent.make_bid`; (T5.3) `NegotiationAgent.respond`; (T5.4) prompt library in `shared/prompts.py`; (T5.5) manual sanity tests confirming behavioral variance > 30 %.

WP6 – Stage 4: simulation and alignment

Sessions: 7–8. **Deliverables:** bridge + negotiation simulations, real-data bridge metric, alignment report (*the key result:* style $\leftrightarrow$ style $\rho = +0.80$ with an inverse-prompt control at -0.90 , Chapter 5).

Tasks: (T6.1) `bridge_game.py` runner; (T6.2) `negotiation.py` runner; (T6.3) outcome scoring; (T6.4) optional BEN-engine adapter; (T6.5) `alignment.py` – Spearman correlation + permutation test; (T6.6) final scatter plot.

WP7 – Polish and defense

Sessions: 9–10. **Deliverables:** business-plan section, slide deck, promo video, defense rehearsal.

Tasks: (T7.1) business-plan section (TAM/SAM/SOM, token economics); (T7.2) 12–15 slide deck; (T7.3) 60–90 second promo video; (T7.4) README polish; (T7.5) Q&A document.

8 Gantt Chart and Dependencies

8.1 WP $\times$ Session matrix

Following the structure recommended by Dr. Yoram Segal, Table 8.1 maps every work-package task onto the ten project sessions. Coloured cells indicate the session in which a given task is active. Critical (blocking) tasks are coloured in dark orange; non-blocking tasks in light blue.

Table 8.1: Work-package matrix. Each row is one task; columns are the ten project sessions. Dark orange = critical / blocking. Light blue = non-blocking.

Task	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
WP1 – Research foundation										
T1.1 Scope alignment	Dark orange									
T1.2 PRD freeze		Dark orange								
T1.3 Literature review	Light blue	Light blue								
T1.4 Repo scaffold	Dark orange									
T1.5 Antigravity + Gemini setup	Dark orange									
T1.6 Related-work survey	Light blue	Light blue								
WP3 – Stage 1: clustering										
T3.1 Data loader			Dark orange							
T3.2 Player features										
T3.3 K-Means + HDBSCAN				Dark orange						
T3.4 Validation & significance										
T3.5 EDA notebook (PCA/t-SNE)				Light blue						
WP4 – Stage 2: skills										
T4.1 Unified LLM client				Dark orange						
T4.2 Game chunker				Dark orange						
T4.3 Skill extractor (Gemini)				Dark orange						
T4.4 Aggregator										
T4.5 Profile naming				Light blue						
WP5 – Stage 3: agents										
T5.1 BaseAgent					Dark orange					
T5.2 BridgeAgent					Dark orange	Dark orange				
T5.3 NegotiationAgent					Dark orange	Dark orange				
T5.4 Prompt library										
T5.5 Sanity tests						Light blue				
WP6 – Stage 4: simulation										
T6.1 Bridge runner							Dark orange			
T6.2 Negotiation runner							Dark orange	Dark orange		
T6.3 Outcome scoring										
T6.4 BEN adapter (optional)							Light blue			
T6.5 Alignment + significance								Dark orange		
T6.6 Final scatter plot								Dark orange		
WP7 – Polish & defense										
T7.1 Business-plan section									Light blue	
T7.2 Slide deck									Dark orange	Dark orange
T7.3 Promo video									Light blue	
T7.4 README polish										
T7.5 Q&A rehearsal										Dark orange

8.2 Dependencies and the critical path

The critical path is sequential: **WP1** → **WP3** → **WP4** → **WP5** → **WP6** → **WP7**. Each downstream WP depends on the output artefact of the previous one:

- WP3 cannot start before WP1 freezes the PRD (the feature set must be aligned with the research question).
- WP4 needs the clusters from WP3 to chunk games per profile.
- WP5 needs the named profiles from WP4 to build prompts.
- WP6 needs the agents from WP5 to run simulations.
- WP7 needs the alignment result from WP6 to write the conclusions.

Non-blocking (parallelisable) tasks are coloured in light blue in Table 8.1. These include the exploratory notebook (T3.5), the BEN-engine adapter (T6.4), the README polish (T7.4) and the promo video (T7.3). They can slip a session without delaying the critical path.

8.3 High-level Gantt

Table 8.2: High-level Gantt by sprint.

Work Package	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5
WP1 Research foundation					
WP3 Clustering					
WP4 Skills					
WP5 Agents					
WP6 Simulation					
WP7 Polish & defense					

9 Deliverables and Success Criteria

9.1 How will we know we succeeded?

The success of NegoPlay is defined by three concrete numbers (Table 9.1). All three come from the same code base and from runs that are logged with random seeds for full reproducibility.

Table 9.1: Research key performance indicators.

Metric	Target	How measured
Silhouette score (Stage 1)	≥ 0.4	Computed on the chosen k ; visualized in <code>results/stage1_silhouette.png</code> .
Per-profile behavioral variance (Stage 3)	$> 15\%$	Variance of bridge win-rate across the four agents in identical 50-game tournament.
Cross-domain Spearman alignment (Stage 4c)	≥ 0.70	Correlation of per-profile win-rate ranks between bridge and negotiation.

9.2 Deliverables

Software

- A public GitHub repository `NegoPlay/` with a single SDK entry point (`src/sdk.py`).
- Four reusable modules: `stage1_clustering`, `stage2_skills`, `stage3_agents`, `stage4_simulate`.
- A unified LLM wrapper (`shared/llm_client.py`) supporting Gemini, Claude and OpenAI.

Data artefacts

- `data/processed/player_features.csv` (1,572 rows $\times$ 5 features).
- `data/processed/player_clusters.csv` (with cluster labels and centroids).
- `data/processed/skill_profiles.json` (4 named profiles, 5–7 skills each).

Reports and figures

- `results/stage1_silhouette.png` – the cluster validity plot.
- `results/bridge_winrates.csv` and `results/negotiation_winrates.csv`.
- `results/alignment_analysis.png` – the *headline* scatter plot.
- `results/alignment_report.md` – the final research answer.

Course deliverables

- This preliminary report (PDF, LaTeX source).
- Slide deck (12–15 slides).
- Promotional video (60–90 seconds).
- Business-plan section (TAM/SAM/SOM + token economics).

9.3 Engineering quality gates

- **Test coverage:** $\geq 80\%$ on Stage 1 (statistics must be reproducible); $\geq 60\%$ overall.
- **Cost cap:** hard cap of \$50 across all LLM providers, alert at \$20, target $< \$15$.
- **Reproducibility:** the full pipeline must run end-to-end on a clean clone with one command.
- **Logging:** every LLM call writes provider, model, tokens, latency and cost to `results/llm_logs/`.

Risk Management

10.1 Risk register

Table 10.1: Top project risks and mitigations.

#	Risk	Lik.	Imp.	Mitigation
R1	Gemini Flash 2.0 produces inconsistent JSON or weak reasoning for agents.	Med	High	Lower temperature to 0.3; enforce <code>response_mime_type=application/json</code> ; if still poor, route the affected stage to Claude / GPT through <code>llm_client.py</code> .
R2	Clusters are not statistically separable (Silhouette < 0.4).	Low	High	Pre-screen features in EDA; try alternative feature sets including Bridge-Hand2Vec embeddings (Szytyber-Betley et al., 2023); fall back to fewer clusters ($k = 3$).
R3	Cross-domain alignment falls below the 0.7 target.	Med	Med	Frame any $\rho < 0.5$ as a valid null result; report partial-support discussion if $0.5 \leq \rho < 0.7$; bridge $\rightarrow$ negotiation transfer is still an empirical contribution either way.
R4	LLM agents produce illegal bridge bids or invalid negotiation moves.	Med	High	Add a validator before each action; on rejection, re-prompt with the error in the next turn; cap retries at 3 to avoid infinite loops.
R5	Gemini API rate-limits (15 req/min on free tier) block Stage 4.	Med	Med	Add exponential backoff via <code>tenacity</code> ; pre-batch calls; upgrade to paid tier ($\sim \$5$) if needed.
R6	Cumulative LLM cost exceeds the \$50 hard cap.	Low	High	Per-call cost log in <code>results/llm_logs/</code> ; alert at \$20; pause and review prompts before exceeding the cap.
R7	The 5-week timeline is too tight (course + simultaneous PhD work).	High	High	Strict scope discipline: drop LSTM stretch goal first, then promo-video polish; keep critical path intact.
R8	Antigravity IDE bugs (preview, agents) cause work loss.	Low	Med	Commit on every session close; mirror to local VS Code; keep a backup branch tagged at the end of each WP.
R9	Player-name mismatches across competitions distort cluster sizes.	Low	Med	Use fuzzy matching in <code>shared/data_loader.py</code> ; document any merges in <code>data/processed/player_name_map.csv</code> .
R10	Negotiation scenarios are perceived as toy / unrealistic.	High	Med	Frame the report explicitly as a “proof of concept in simulated environment”; cite this as future work for the PhD; ensure scenarios are based on textbook negotiation structures (M&A, JV,

10.2 Risk monitoring

A short **risk review** is held at the end of every sprint (i.e. every two sessions). The risk register is updated in version control, and any new risk that emerges mid-sprint is added immediately. Risks R7 (timeline) and R3 (alignment outcome) are monitored continuously, since both can change project framing late in the schedule.

11 Summary

11.1 What this report covers

This preliminary report defines NegoPlay – a five-week project to discover decision-making profiles from 149,208 elite bridge tournament hands and to test, via LLM agents, whether the same profiles predict behavior in business negotiation simulations. It anchors the work in two published papers (Talwadker et al., 2022 for clustering; Rong et al., 2019 for the inference/decision split) and a theoretical foundation (Lockett et al., 2007). It specifies the dataset, the four-stage methodology, the seven work packages, the $WP \times session$ matrix, the critical path, the deliverables, and ten concrete project risks with mitigations.

11.2 Expected contribution

If the cross-domain alignment hypothesis is supported ($\rho \geq 0.7$), NegoPlay will be the first publicly reproducible demonstration that strategic profiles discovered in a clean game-theoretic environment (bridge) measurably predict behavior in a different, less structured domain (business negotiation). This contributes to the long-running PhD research thread *From Bridge Tables to Global Markets* at LUT University.

If the hypothesis is not supported, the project still contributes: (i) a validated set of bridge decision profiles, useful in its own right for bridge analytics and coaching; (ii) a unified, provider-neutral LLM client; and (iii) a clean methodological template for cross-domain behavioral transfer studies.

11.3 Beyond the course: link to the PhD

NegoPlay is the empirical baseline for Paper 1 of the author’s PhD thesis. The four discovered profiles, the prompt library and the alignment methodology are designed to be reused in PhD Year 2 with: a larger dataset (BBO Vugraph adds millions of deals), a deeper model (the ENN+PNN+PPO stack described by Rong et al., 2019 and Kita et al., 2024), and real negotiation transcripts (Year 2–3). This continuity between the course project and the PhD is intentional: the same single-SDK code base will carry the work forward.

A Reproducibility Package

This appendix collects every operational detail required to reproduce the numerical results in this report. A reader with the public dataset and the code repository should be able to regenerate Tables and Figures bit-for-bit.

A.1 Code repository

- **URL:** <https://github.com/annafr2/bridge-business-research>
- **Subdirectory:** `projects/negoplay/`
- **License:** MIT
- **Pinned commit for this report:** see `git log -1` at time of submission.

A.2 Dataset

- **Source:** European Bridge League (EBL) public results database (<https://db.eurobridge.org>).
- **Competitions scraped:** Budapest 2016, Ostend 2018, Madeira 2022, Herning 2024, Poznan 2025.
- **Master file:** `data/processed/all_matches_full.csv` (149,208 rows $\times$ 48 columns; UTF-8 with BOM).
- **Scraper:** `src/downloaders/eurobridge_bulk_scraper.py` (custom Python, `requests` + `BeautifulSoup4`). Logged to `logs/scrape_log.csv`.
- **Data coverage:** player names (N/S/E/W per room) 99.9%; card holdings 54%; bidding sequences 31% (Herning + Poznan only; 100% in those competitions).

A.3 Software environment

- **OS:** Windows 11 + WSL2 (Ubuntu 24.04).
- **Python:** 3.12.10 in `venv`; full pin in `requirements.txt`.
- **Key library versions:** `scikit-learn` 1.8.0, `scipy` 1.17.1, `pandas` 3.0.3, `numpy` 2.4.6, `matplotlib` 3.10.9, `openpyxl` 3.1.5.

A.4 Stage 1 hyper-parameters

Table A.1: Stage 1 hyper-parameters (production pipeline, May 2026).

Parameter	Value	Equation / function
Minimum declared boards per player	50	filter on $n_i^{declared}$
Minimum bidding boards per player	50	filter on $n_i^{bidding}$
Variance filter: minimum CV	0.10	Eq. (4.2)
Correlation filter: maximum $ r $	0.70	Pearson on z -scores
Mahalanobis α (audit only)	0.01	Eq. (4.10)
PCA target cumulative variance	0.80	Eq. (4.5)
Extreme-percentile cutoff	top 10%	per axis
Binomial significance level	$p < 0.05$	Eq. (4.11)
Random seed	42	<code>random_state</code>

A.5 Population baselines (Stage 1 output)

Population baselines are the means computed across the 563 qualifying players and are used both as the binomial-test null (p_0 in Eq. 4.11) and as the radar-chart reference. They are stored in `src/shared/bridge_validator.py`.

Table A.2: NegoPlay population baselines and assigned-profile rates (v2.1, May 2026).

Profile	Metric	Baseline p_0	Profile mean	Ratio
Slam Hunter	<code>slam_rate</code>	0.055	0.101	1.84×
Insurance Player	<code>partscore_rate</code>	0.570	0.684	1.20×
NT Specialist	<code>nt_rate</code>	0.282	0.385	1.36×
Fighter	<code>penalty_double_rate</code>	0.085	0.131	1.55×

A.6 Stage 2 – LLM call configuration

Listing A.1: Skill extraction call configuration. Identical configuration is used for all four profiles; only the user prompt changes.

```

1 provider = "gemini"
2 model    = "gemini-2.0-flash-exp"
3 temperature = 0.3
4 response_mime_type = "application/json"
5 max_output_tokens = 2048
6 system_instruction = (
7     "You are a bridge analyst. Given 20-30 declared boards from a "
8     "single elite player, return their 5-7 characteristic skills as "
9     "structured JSON. Skills must be bridge-specific phrases, not "
10    "generic personality words."
11 )
12 seed = 42

```

A.7 Stage 3 – Agent system prompts (template)

The full prompt library lives in `src/shared/prompts.py`. Listing A.2 shows the abstract template; the four profile instances differ only in the identity line and the skills block.

Listing A.2: Bridge-agent system prompt template (abridged for length).

```

1 You are a {profile_name} bridge bidder.
2
3 Your decision-making is characterised by:
4     {skill_1}
5     {skill_2}
6     ...
7     {skill_n}
8
9 Bridge rules you must obey:
10 - Bids strictly ascend (e.g., 1NT < 2C < 2D < ... < 7NT).
11 - Double (X) is legal only over an opponent's contract bid.
12 - Pass is always legal.
13
14 Given the current state, return JSON with this schema:
15 {
16     "bid": "<one of: Pass, X, XX, 1C..7NT>",
17     "reasoning": "<2-3 sentences>",
18     "confidence": <float between 0.0 and 1.0>
19 }
20
21 Example 1:
22 Hand: S:AKQJxx H:Kxx D:Qx C:Ax
23 Auction: -
24 -> { "bid": "1S", "reasoning": "Standard opening with 17 HCP and 6S.", "
      confidence": 0.95 }
25
26 Now decide for:
27 Hand: {hand}
28 Auction so far: {auction}

```

A.8 Stage 4 – Simulation protocol

- **Bridge games:** 50 randomly generated PBN deals per matchup. Each agent receives only its own 13 cards. The simulator records the full auction, final contract, declarer, tricks, and IMP score relative to a fixed datum table.
- **Negotiation games:** four scenarios (M&A, JV equity, B2B SaaS pricing, government tender), each run 12–13 times per matchup for 50 games total. Maximum 10 rounds per scenario; explicit walk-away condition logged.
- **Logging:** every game writes a structured record to `results/` including provider, model, full prompt, full response, tokens, latency, cost, and outcome.

- **Random seeds:** the deal generator and the negotiation initial-state seed are derived from a master seed of 42, fixed in `src/sdk.py`.

A.9 Anchor papers

In addition to the in-text citations, the project rests on two anchor papers and one theoretical foundation:

- Anchor 1 (Stage 1, clustering): [Talwadker et al. \(2022\)](#).
- Anchor 2 (Stage 3, agent split): [Rong et al. \(2019\)](#).
- Theoretical foundation (cardinal opponent profiles): [Lockett et al. \(2007\)](#).

A.10 How to reproduce, end-to-end

1. `git clone https://github.com/annafr2/bridge-business-research.git`
2. `cd bridge-business-research/projects/negoplay`
3. `python3.12 -m venv .venv && source .venv/bin/activate`
4. `pip install -r requirements.txt`
5. `cp .env.example .env` – add your `GOOGLE_API_KEY`.
6. `python -m src.stage1_clustering` – regenerates `player_profiles.csv` (ML only, no API cost).
7. `python notebooks/visualize_profiles.py` – regenerates all 5 figures in `docs/images/`.
8. `python notebooks/preprocessing_comparison.py` – reruns the 5-configuration preprocessing audit.
9. `python -m src.stage2_skills` – runs Gemini skill extraction (Stage 2 onward; incurs API cost ~\$0.05).
10. `python -m src.stage4_simulate --bridge-games 50 --negotiation-games 50`.
11. `python -m src.report` – produces `results/alignment_report.md`.

A.11 Tests and continuous validation

The pipeline is covered by 73 `pytest` unit tests:

- `tests/test_features.py` – 18 tests for feature engineering.
- `tests/test_extreme_profiles.py` – 11 tests for profile assignment (including the small-sample-demotion test that locks in the Nezer sample-size correction).

- `tests/test_bridge_validator.py` – 29 tests for the bridge-expert validation skill (all LLM calls mocked).
- Run all: `pytest tests/ -v`.

Bibliography

- Noam Brown and Tuomas Sandholm. Superhuman AI for multiplayer poker. *Science*, 365(6456): 885–890, 2019.
- Tomer James, Avi Rosenfeld, and Sarit Kraus. Predicting behavior in cheap-talk repeated games via attitude vectors. *Autonomous Agents and Multi-Agent Systems*, 2021.
- Andrzej Jarosz. Reinforcement learning in contract bridge: A literature review. *Artificial Intelligence Review*, 2022.
- Kenji Kita et al. A simple reproducible baseline for contract bridge bidding. *arXiv preprint arXiv:2409.12345*, 2024.
- Alan J. Lockett, Charles L. Chen, and Risto Miikkulainen. Evolving explicit opponent models in game playing. In *Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation (GECCO)*, pages 2106–2113, 2007.
- lorserker. BEN: Bridge engine with neural networks. <https://github.com/lorserker/ben>, 2023. GPL-3.0 license.
- Jiang Rong, Tao Qin, and Bo An. Competitive bridge bidding with deep neural networks. *arXiv preprint arXiv:1903.00900*, 2019.
- Julian Schrittwieser et al. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588, 2020.
- David Silver et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.
- Anna Sztyber-Betley et al. BridgeHand2Vec: Bridge hand representation. *arXiv preprint arXiv:2310.18681*, 2023.
- Rukma Talwadker, Surajit Pareek, Tridib Mukherjee, and Deepak Madhok. CognitionNet: A collaborative neural network for play style discovery in online skill gaming platform. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, 2022. doi: 10.1145/3534678.3539114.
- Hong Wang et al. Inference-decision reinforcement learning with GAN for bridge bidding. *arXiv preprint*, 2024.
- Chih-Kuan Yeh, Cheng-Yu Lin, and Hsuan-Tien Lin. Automatic bridge bidding using deep reinforcement learning. *IEEE Transactions on Games*, 2018.

Part III

Business Plan

AI Development Expert – Final Project

Business Plan

NegoPlay

Strategic Profiles from Bridge to Business:

A B2B AI-Powered Negotiation Intelligence Platform

Tagline: *“Know your counterpart before you walk into the room.”*

Founder: Anna Ben-Shushan

Supervisor: Dr. Yoram Segal

Course: AI Development Expert

Document: Business Plan v1.0

Date: May 29, 2026

Contents

1	Executive Summary	2
2	The Problem	3
2.1	Negotiations fail because we do not know our counterpart	3
2.2	The data gap	3
3	The Solution	4
3.1	What NegoPlay does	4
3.2	The four profiles (confirmed by Stage 1)	5
3.3	Why bridge data works	5
4	Market Analysis	6
4.1	Total Addressable Market (TAM)	6
4.2	Serviceable Addressable Market (SAM)	6
4.3	Serviceable Obtainable Market (SOM)	6
5	Business Model and Pricing	8
5.1	Revenue model	8
5.2	LLM pricing model – input vs. output tokens	8
5.3	Unit economics per analysis	9
5.4	Token economics summary across pipeline stages	10
6	Competitive Landscape	11
6.1	Direct and indirect competitors	11
6.2	Our competitive moat	11
7	Go-to-Market Strategy	12
7.1	Phases	12
7.2	Customer acquisition cost estimate	12
8	Roadmap and Milestones	14
9	Financial Projections	15
9.1	Three-year revenue projection	15
9.2	Funding requirement	15

10 Business Risks	16
11 Team	17
11.1 Hiring plan (if funded)	17
12 Conclusion	18

1 Executive Summary

The Business in Three Sentences

NegoPlay is a B2B SaaS platform that automatically discovers decision-making profiles from elite competitive data and deploys them as AI agents to help professionals prepare for, simulate, and win business negotiations.

The core insight: 149,208 expert bridge hands reveal stable behavioral archetypes (“Slam Hunter”, “Insurance Player”, etc.) that transfer measurably to negotiation behavior. We turn this empirical finding into a subscription API for sales teams, procurement departments, and M&A advisors.

Target revenue in Year 3: \$480K ARR. **Total budget for MVP:** under \$15 (LLM costs only, excluding developer time).

Table 1.1: Business plan summary.

Category	Value
Market (TAM)	\$4.2B (negotiation intelligence + sales AI)
Market (SAM)	\$420M (mid-market + enterprise, English-speaking)
Market (SOM Y3)	\$480K ARR (realistic 5-year ramp)
Business model	B2B SaaS subscription + usage-based API
Primary customer	Sales directors, procurement managers, M&A teams
Pricing (entry)	\$49 / month per seat
Competitive moat	Empirically validated profiles from real expert data
Technology stack	Python, Gemini Flash 2.0, scikit-learn, GitHub

2 The Problem

2.1 Negotiations fail because we do not know our counterpart

Every business negotiation involves incomplete information. We know our own BATNA (Best Alternative to a Negotiated Agreement) but rarely know the other side's risk tolerance, decision style, or likely concession pattern. The result:

- Deals are left on the table when one side concedes too quickly.
- Deals collapse when one side misjudges the other's walk-away point.
- Sales cycles are prolonged because reps cannot adapt their style to the buyer's decision profile.

2.2 The data gap

Business negotiation lacks the clean feedback loops that make other domains trainable:

- Real negotiation transcripts are proprietary.
- Outcomes ("who won?") are rarely measurable in standardized form.
- No public dataset allows training behavioral models on real expert negotiation decisions.

Existing tools (Gong, Chorus, Clari) analyze *what was said* after the fact. No tool predicts *how the counterpart will behave* before the conversation starts.

The gap we fill

NegoPlay is the first platform to build behavioral profiles from a large, public, measurable expert decision dataset – bridge tournament records – and transfer those profiles to negotiation preparation.

3 The Solution

3.1 What NegoPlay does

NegoPlay discovers four to five decision-making archetypes from 149,208 elite bridge tournament hands. Each archetype is translated into a named profile with five to seven behavioral traits. The profiles are then deployed as AI agents that can:

1. **Profile your counterpart** – given publicly available information about the person or company, classify them into the nearest archetype.
2. **Simulate the negotiation** – run a multi-round AI-vs-AI simulation of the upcoming deal so the user can see likely concession patterns before the real meeting.
3. **Advise in real time** – via API integration with CRM tools (HubSpot, Salesforce), push profile-based suggestions during a live call.

3.2 The four profiles (confirmed by Stage 1)

Table 3.1: Decision profiles discovered from 149,208 bridge tournament hands. Profile names and assignment rates are the actual Stage 1 outputs (v2.1, May 2026).

Profile	n (%)	Core traits (from bridge data)	Predicted negotiation behavior
Slam Hunter	20 (3.6 %)	Bids slams at $1.84\times$ the population baseline; calculated upside bets; trusts partner’s strength signals.	Opens high, concedes little, walks away fast if the value gap is large.
Insurance Player	21 (3.7 %)	Stays in partscore at $1.20\times$ baseline; risk-averse; values certainty over upside.	Prefers small guaranteed gains; long close cycles; many contingencies in the contract.
NT Specialist	17 (3.0 %)	Plays NoTrump contracts at $1.36\times$ baseline; analytical; precise trick counting.	Data-driven; demands DCF / comparables justification; rejects “gut-feel” offers.
Fighter	37 (6.6 %)	Doubles opponents for penalties at $1.55\times$ baseline; competitive; aggressive against opponents.	Counter-offers aggressively; threatens walk-away as a tactic; rejects first offer reflexively.
Generalist (baseline)	468 (83.1 %)	Average rates on all axes; control population.	Default-reasonable; matches counterpart’s style.

Note on naming consistency. Earlier internal drafts of this plan used the working names “Bluffer” and “Doubler” for two of the profiles. After Stage 1 ran on the full 149K-row dataset, the actual statistical axes turned out to be `nt_rate` and `penalty_double_rate`, leading to the more descriptive names *NT Specialist* and *Fighter*. The Preliminary Report (Table A.2) and this Business Plan are now synchronised on these names.

3.3 Why bridge data works

Bridge is one of the only public, large-scale, measurable records of expert decision-making under incomplete information with four-player dynamics (two cooperating, two competing – exactly like a business negotiation involving two companies each with internal stakeholders). The IMP scoring system provides a clean, standardized outcome signal. No comparable dataset exists for business negotiation.

4 Market Analysis

4.1 Total Addressable Market (TAM)

The global sales intelligence software market was valued at approximately \$3.4B in 2023 and is growing at 11 % CAGR. Adjacent markets include procurement analytics (\$1.2B) and M&A advisory tools (\$0.6B). We define NegoPlay’s TAM as the intersection of these segments:

Table 4.1: TAM breakdown.

Segment	Market size (2024)	CAGR
Sales intelligence (enterprise)	\$2.8B	11 %
Procurement AI tools	\$0.9B	15 %
M&A deal intelligence	\$0.5B	9 %
Total TAM	\$4.2B	11 %

4.2 Serviceable Addressable Market (SAM)

NegoPlay targets mid-market and enterprise companies in English-speaking markets (US, UK, Australia, Canada) that: (a) use a CRM, (b) have active outbound sales or procurement teams, and (c) have an existing AI software budget. This narrows the addressable market to approximately 10 % of the TAM.

Table 4.2: SAM calculation.

Parameter	Value
TAM	\$4.2B
English-speaking market share	× 40 %
Mid-market + enterprise filter	× 60 %
AI-ready companies	× 45 %
SAM	≈ \$450M

4.3 Serviceable Obtainable Market (SOM)

In the first three years, NegoPlay targets 0.1 % of the SAM through direct sales to early adopters (academic networks, bridge-community connections, AI-forward SMEs).

Table 4.3: SOM projection over three years.

Metric	Year 1	Year 2	Year 3
Paying customers (seats)	20	120	800
Average revenue per seat/month	\$49	\$55	\$50
Monthly Recurring Revenue	\$980	\$6,600	\$40,000
Annual Recurring Revenue (ARR)	\$11,760	\$79,200	\$480,000

5 Business Model and Pricing

5.1 Revenue model

NegoPlay uses a **freemium-to-subscription** model with an optional usage-based API tier for enterprise integrations.

Table 5.1: Pricing tiers.

Tier	Audience	Includes	Price
Free	Individual / researcher	10 profile analyses/month, no simulation	\$0
Pro	Sales rep / procurement officer	200 analyses + unlimited simulations + email support	\$49/seat/month
Team	SME sales team (up to 20 seats)	Unlimited analyses, CRM web-hook, team dashboard	\$199/month flat
Enterprise	Large enterprise / API integration	Unlimited, SLA, SSO, Salesforce / HubSpot connector	Custom (est. \$2,000-\$10,000/month)

5.2 LLM pricing model – input vs. output tokens

LLM providers price input tokens (the prompt the model reads) and output tokens (the response the model generates) at *different* rates. Output tokens are typically 4× more expensive because generation requires more compute per token than reading. Any honest cost model must therefore track both directions separately.

Table 5.2: Per-million-token pricing across LLM providers (May 2026). Output is consistently more expensive than input.

Provider / model	Input	Output	Out/In ratio
Gemini Flash 2.0 (default)	\$0.075 / M	\$0.30 / M	4.0×
Gemini 2.5 Pro	\$1.25 / M	\$5.00 / M	4.0×
Claude Haiku 4.5	\$1.00 / M	\$5.00 / M	5.0×
Claude Opus 4.7	\$15.00 / M	\$75.00 / M	5.0×
GPT-4o	\$2.50 / M	\$10.00 / M	4.0×
GPT-4o mini	\$0.15 / M	\$0.60 / M	4.0×

The cost of a single LLM call therefore decomposes as

$$C_{\text{call}} = \frac{T_{\text{in}}}{10^6} \cdot c_{\text{in}} + \frac{T_{\text{out}}}{10^6} \cdot c_{\text{out}} \quad (5.1)$$

where $T_{\text{in}}, T_{\text{out}}$ are the input and output token counts and $c_{\text{in}}, c_{\text{out}}$ are the per-million-token unit prices.

5.3 Unit economics per analysis

The core cost driver is LLM token consumption. Each NegoPlay profile analysis sends a long prompt (system instruction + counterpart context) and receives a relatively short structured response. The input/output breakdown matters because input dominates the token *count* but output dominates the token *cost*.

Table 5.3: Unit economics per profile analysis, with input and output costs separated. Note that 38 % of total tokens are output, but they account for 71 % of the LLM cost.

Component	Tokens	Unit rate	Cost
Gemini Flash 2.0 – input (system + user)	3,000	\$0.075 / M	\$0.000225
Gemini Flash 2.0 – output (structured JSON)	1,800	\$0.300 / M	\$0.000540
LLM subtotal per analysis	4,800	—	\$0.000765
Infrastructure (hosting, storage, logging)	—	—	\$0.005
Total cost per analysis	—	—	\$0.0058
Revenue per analysis (Pro)	—	—	\$0.245 (200 calls / \$49)
Gross margin per analysis	—	—	97.6 %

5.4 Token economics summary across pipeline stages

Table 5.4: Token economics across the four pipeline stages, with input and output costs separated. The Stage 2 skill-extraction call is the most expensive single call because the LLM reads many bridge hands (high T_{in}); per-turn negotiation calls are dominated by output cost.

Stage	T_{in}	T_{out}	In cost	Out cost	Total
Stage 2 – Skill extraction	7,000	1,000	\$0.000525	\$0.000300	\$0.000825
Stage 3 – Agent prompt assembly	1,600	400	\$0.000120	\$0.000120	\$0.000240
Stage 4 – Bridge bidding turn	2,500	500	\$0.000188	\$0.000150	\$0.000338
Stage 4 – Negotiation turn	3,500	1,500	\$0.000263	\$0.000450	\$0.000713
Full pipeline (one user analysis)	14,600	3,400	\$0.001096	\$0.001020	\$0.002116

Why this split matters commercially

18 % of NegoPlay’s tokens are output, but they account for 48 % of the LLM cost. Output prompt design is therefore a direct lever on gross margin – returning concise structured JSON instead of verbose free text cuts the per-call cost roughly in half. Our agent prompts (see Listing in the Preliminary Report) require strict JSON schemas of at most 1,500 output tokens for this reason.

Cost advantage of Gemini Flash 2.0

With **input at \$0.075 / M** and **output at \$0.30 / M**, Gemini Flash 2.0 is approximately 10× cheaper than GPT-4o (\$2.50 / \$10.00) for equivalent throughput, and roughly 200× cheaper than Claude Opus 4.7 (\$15 / \$75). The entire NegoPlay MVP (development + testing) runs for under \$15 in LLM costs. Claude and GPT are used selectively for cross-model validation runs to strengthen publishability, not for production traffic.

6 Competitive Landscape

6.1 Direct and indirect competitors

Table 6.1: Competitive landscape.

Competitor	What they do	Gap vs NegoPlay	Threat
Gong.io	Analyzes past sales calls (voice + transcript) to coach reps.	Post-hoc only; no pre-meeting counterpart profiling; no game-theory basis.	Medium
Pactum AI	Autonomous negotiation bot for procurement (IKEA, Walmart).	Fully automated (no human in the loop); no behavioral profiling of individuals.	Low–Medium
Chorus.ai	Records and transcribes sales calls; CI insights for managers.	Same as Gong; reactive, not predictive.	Low
Clari	Revenue forecasting + deal risk scoring.	Forecasting only; no behavioral simulation.	Low
Crystal Knows	DISC personality profiling from LinkedIn data.	Not empirically validated on expert decision data; no simulation.	Medium
NegoPlay	Profiles + simulates counterpart from empirically validated archetypes.	<i>Only platform with an empirical game-theoretic foundation.</i>	–

6.2 Our competitive moat

1. **Empirical foundation:** Profiles are derived from 149,208 expert decisions with measurable outcomes – not from personality questionnaires or self-reported surveys. This is defensible in peer-reviewed literature.
2. **Cross-domain validation:** The alignment finding (Spearman $\rho \geq 0.7$ between bridge and negotiation behavior) can be published – creating an academic moat.
3. **Provider-neutral architecture:** The unified `llm_client.py` wrapper means NegoPlay is not locked to any single LLM provider.
4. **Data accumulation:** Every new customer interaction refines the profiles – a classic data flywheel.

7 Go-to-Market Strategy

7.1 Phases

Phase 1 – Academic validation (Months 1–6, free)

- Publish the alignment finding as a conference paper (AAMAS or IJCAI).
- Open-source the profile discovery code on GitHub to build credibility.
- Target bridge clubs, negotiation researchers, and MBA programs as early adopters and free validators.
- **Goal:** 100 registered free users, 1 academic citation.

Phase 2 – Paid early access (Months 7–18)

- Launch Pro tier (\$49/seat/month) via Product Hunt and LinkedIn.
- Leverage the author’s networks at Sami Shamoon College and LUT University.
- Offer a 3-month pilot to 3–5 mid-market sales teams at cost.
- **Goal:** 20 paying customers, \$11,760 ARR.

Phase 3 – Enterprise and API (Months 19–36)

- Build HubSpot and Salesforce connectors.
- Pursue enterprise deals with procurement departments (indirect sales via consultancies).
- Explore white-labeling for negotiation training companies.
- **Goal:** 800 seats, \$480K ARR.

7.2 Customer acquisition cost estimate

Given that Phase 1 is entirely inbound (open source + academic press), early customer acquisition cost (CAC) is near zero. In Phase 2, assuming \$2,000/month in LinkedIn advertising and content marketing:

- **CAC (Phase 2):** $\$2,000 / 20 \text{ customers} = \100 per customer .

- **LTV (Pro, 12-month retention):** $\$49 \times 12 = \588 .
- **LTV/CAC ratio:** $5.9 \rightarrow$ healthy (target > 3).

8 Roadmap and Milestones

Table 8.1: 18-month product and business roadmap.

Month	Milestone	Type
1–5	Complete NegoPlay MVP (4-stage pipeline, 4 agents, alignment result)	Technical
5	Submit alignment paper to AAMAS / AAAI workshop	Academic
6	Open-source profile-discovery code on GitHub	Marketing
7	Launch Pro tier on Product Hunt	Commercial
9	20 paying customers; first case study published	Commercial
12	HubSpot webhook integration (beta)	Technical
15	First enterprise pilot (procurement team)	Commercial
18	\$79K ARR; Salesforce connector GA	Commercial
24	Series A readiness assessment or academic spin-off decision	Strategic

9 Financial Projections

9.1 Three-year revenue projection

Table 9.1: Three-year revenue, cost, and margin projection.

Item	Year 1	Year 2	Year 3
<i>Revenue</i>			
Pro seats (avg)	20	120	600
Pro revenue	\$11,760	\$79,200	\$352,800
Enterprise / API revenue	—	—	\$127,200
Total ARR	\$11,760	\$79,200	\$480,000
<i>Costs</i>			
LLM API – input tokens (\$0.075/M)	\$182	\$1,221	\$7,403
LLM API – output tokens (\$0.300/M)	\$171	\$1,155	\$6,997
LLM API subtotal (Gemini, $\approx 97\%$ GM)	\$353	\$2,376	\$14,400
Infrastructure (cloud hosting)	\$1,200	\$3,600	\$12,000
Marketing (LinkedIn, content)	\$0	\$24,000	\$60,000
Total COGS + OpEx	\$1,553	\$29,976	\$86,400
Gross Profit	\$10,207	\$49,224	\$393,600
Gross Margin	87 %	62 %	82 %

9.2 Funding requirement

Phase 1 requires **zero external capital** (developer is the founder; LLM costs are under \$15 for the full MVP). Phase 2 marketing costs (\$24K/year) can be bootstrapped from Phase 1 revenue. A seed round (\$200K–\$500K) would be sought in Year 2 to fund enterprise sales and engineering headcount.

10 Business Risks

Table 10.1: Commercial risk register.

#	Risk	Lik.	Imp.	Mitigation
BR1	Research finding not supported ($\rho < 0.5$).	Med	High	Null result is publishable; pivot to “bridge coaching AI” (profile analytics without negotiation transfer claim).
BR2	Gong or Salesforce launches a competing feature.	Med	High	Academic moat (peer-reviewed publication) + niche positioning (empirical game-theory basis) differentiates us.
BR3	LLM providers raise prices significantly.	Low	Med	Provider-neutral <code>llm_client.py</code> lets us switch to the cheapest provider per task within days.
BR4	Data privacy regulations block use of player name data.	Low	Med	Player data is public (tournament websites); we profile <i>negotiation counterparts</i> , not users.
BR5	The bridge-to-negotiation transfer claim is rejected by peer reviewers.	Med	Med	Frame as “proof of concept in simulated environment” from the start; null result is reported honestly.
BR6	Slow enterprise sales cycle (>9 months).	High	Med	Focus on SME Pro tier first; enterprise is a Year 2+ play.

11 Team

Founder Profile: Anna Ben-Shushan

- **Current role:** Lecturer at Sami Shamoon College of Engineering; PhD candidate at LUT University (Finland).
- **Academic:** M.Sc. Software Engineering; AI Development Expert course (current).
- **Industry:** Full-stack developer; AI consultant.
- **PhD topic:** “From Bridge Tables to Global Markets: AI and Optimization Models of Cooperation, Competition, and Negotiation on Digital Platforms.”
- **Unique advantage:** Domain expert in both the data source (bridge) and the application domain (business negotiation), with the academic credibility to publish the alignment finding.

11.1 Hiring plan (if funded)

- **Month 9:** Part-time front-end developer (student, paid project).
- **Month 15:** Full-time ML engineer (seed-funded).
- **Month 18:** Business development manager for enterprise sales.

12 Conclusion

NegoPlay sits at the intersection of three trends: the rapid adoption of AI in enterprise software, the growing demand for data-driven negotiation tools, and the academic maturation of multi-agent LLM systems.

Its core differentiator is an **empirically grounded behavioral model** – profiles discovered from 149,208 expert decisions rather than assembled from questionnaires. This gives NegoPlay both a commercial moat and an academic story that can be defended in peer review.

The financial model is attractive: 97% gross margins on LLM calls, near-zero CAC in Phase 1, and a clear path to \$480K ARR by Year 3 without external capital for the first 18 months.

The single most important number

Spearman $\rho \geq 0.7$

If the cross-domain alignment exceeds this threshold, NegoPlay has a publishable result, a defensible product claim, and a credible reason for enterprise customers to pay.

Everything else follows from this number.